

# Deep Learning

## DES FONDAMENTAUX AUX MODÈLES AVANCÉS

Architectures, entraînement, optimisation  
et bonnes pratiques en Python



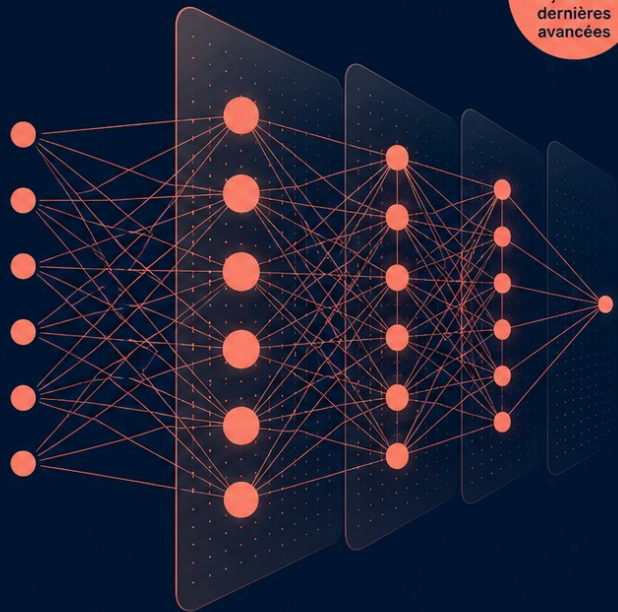
CONCEPTS  
CLÉS



EXEMPLES  
EN PYTHON



APPLICATIONS  
CONCRÈTES



À jour des  
dernières  
avancées

“ La meilleure façon de prédire l'avenir,  
c'est de le construire.

FORMATION IA - DEEP LEARNING

# Deep Learning

Neurones, CNN, transformers et limites :  
comprendre le deep learning simplement.

DanielCraft - Livre debutant

# Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

## 1. Chapitre 1 - Salut, c'est quoi le deep learning ?

- [Ce que ce n'est pas](#)
- [Fil rouge : Ines](#)
- [Ce que tu vas savoir faire](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Deep learning vs ML classique](#)
- [Ce que "50 pages" doivent changer chez toi](#)

## 2. Chapitre 2 - Un neurone : une petite decision ponderee

- [Ce que ce n'est pas](#)
- [Pourquoi des poids ?](#)
- [Limite d'un seul neurone](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Parametres et capacite](#)
- [Scene Ines](#)
- [Trois nombres pour sentir](#)
- [Du neurone au reseau \(teaser\)](#)

## 3. Chapitre 3 - Couches : empiler des transformations

- [Ce que ce n'est pas](#)
- [Profondeur et largeur](#)
- [Fully connected vs specialise](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Representations utiles](#)
- [Scene Ines](#)
- [Lire une architecture comme un brief](#)
- [Connexions residuelles \(aperçu\)](#)

## 4. Chapitre 4 - Activations : introduire du non-lineaire

- [Ce que ce n'est pas](#)
- [En sortie](#)
- [Softmax en sortie \(intuition\)](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)

- [A toi](#)
- [Lien avec la backprop](#)
- [Scene DanielCraft](#)
- [ReLU vs saturations \(terrain\)](#)
- [Activation et lecture metier](#)

## 5. [Chapitre 5 - Backpropagation : l'idee simple de l'apprentissage](#)

- [Ce que ce n'est pas](#)
- [Epochs et batches](#)
- [Optimizers \(aperçu\)](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Ce que tu retiens sans maths](#)
- [Scene Ines](#)
- [Loss : le critere que tu choisis](#)
- [Mini-batch : pourquoi pas tout d'un coup](#)

## 6. [Chapitre 6 - CNN : voir par filtres locaux](#)

- [Ce que ce n'est pas](#)
- [Usages](#)
- [Data augmentation utile](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Lien avec la suite](#)
- [Partage de poids : economie et invariances](#)
- [De la carte de features a la decision](#)
- [Detection et segmentation \(aperçu\)](#)

## 7. [Chapitre 7 - RNN : sequences dans le temps \(overview\)](#)

- [Ce que ce n'est pas](#)
- [Ou les situer en 2026](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Series temporelles](#)
- [Pont vers les transformers](#)
- [LSTM / GRU en une image](#)
- [Split temporel : scene concrete](#)

## 8. [Chapitre 8 - Transformers : l'attention \(overview\)](#)

- [Ce que ce n'est pas](#)
- [Encodeurs, decodeurs, seq2seq](#)

- Contexte et complexite
- Petite histoire
- Lien pratique
- Erreur classique
- En vrai
- A toi
- Du moteur au produit
- Multi-tetes : plusieurs regards
- Pourquoi le GPU aime ca

## 9. Chapitre 9 - Overfitting en deep learning

- Ce que ce n'est pas
- Remedes courants
- Courbes a surveiller
- Generalisation dans le vrai monde
- Petite histoire
- Erreur classique
- En vrai
- A toi
- Capacite et humilite
- Dropout et weight decay (intuition)
- Overfitting au protocole

## 10. Chapitre 10 - GPU : pourquoi ca va plus vite (idee)

- Ce que ce n'est pas
- Ce que ca change pour toi
- Inference vs training
- Limites memoire
- Petite histoire
- Erreur classique
- En vrai
- A toi
- Energie et argent
- Cloud, Colab, API : trois portes
- Precision et quantization (aperçu)

## 11. Chapitre 11 - Transfer learning : reutiliser un cerveau deja forme

- Ce que ce n'est pas
- Strategies
- Domaine shift
- Petite histoire
- Limites
- Erreur classique
- En vrai
- A toi
- Prompting, RAG, fine-tune
- LoRA et adaptation legere (idee)

- Licence et provenance
- Ordre des leviers (rappel)

## 12. Chapitre 12 - Lien avec les LLM : du neurone au chat

- Ce que ce n'est pas
- Ce que tu reconnais maintenant
- Entraîner vs utiliser
- Alignement (idée)
- Petite histoire
- Ethique et coût
- Erreur classique
- En vrai
- A toi
- Pont vers les ateliers
- Température mentale
- Agents : boucler avec freins

## 13. Chapitre 13 - A retenir

- Ce que ce n'est pas
- Petite histoire
- Pont vers la pratique
- Erreur classique
- En vrai
- A toi
- Anti-patterns à garder en tête
- Suite immédiate
- Checklist orale express
- Ce que tu peux déjà faire demain

## 14. Chapitre 14 - Atelier : intuition neurone et couches

- Ce que ce n'est pas
- Exercices (dans l'ordre)
- Petite histoire
- Erreur classique
- En vrai
- Livrable
- A toi
- Variante Ines
- Grille d'auto-évaluation
- Piège du vocabulaire savant

## 15. Chapitre 15 - Atelier : CNN et transfer learning (plan de projet)

- Ce que ce n'est pas
- Étapes (dans l'ordre)
- Petite histoire
- Erreur classique
- En vrai

- [Livrable](#)
- [A toi](#)
- [Conseil DanielCraft](#)
- [Exemple Ines \(rempli partiellement\)](#)
- [Revue par un pair](#)

#### 16. [Chapitre 16 - Atelier : transformer et usage LLM](#)

- [Ce que ce n'est pas](#)
- [Etapas \(dans l'ordre\)](#)
- [Petite histoire](#)
- [Budget tokens](#)
- [Erreur classique](#)
- [En vrai](#)
- [Livrable](#)
- [A toi](#)
- [Variante Max](#)
- [Exemple de mini-RAG \(modele\)](#)
- [Grille d'evaluation rapide](#)

#### 17. [Chapitre 17 - Choisir une architecture \(carte de decision\)](#)

- [Ce que ce n'est pas](#)
- [Questions de decision](#)
- [Anti-patterns](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Pipelines mixtes](#)
- [Lien avec limites et bonnes pratiques](#)
- [Scene de reunion](#)
- [Matrice express \(a recopier\)](#)
- [Quand mixer](#)

#### 18. [Chapitre 18 - Limites du deep learning](#)

- [Ce que ce n'est pas](#)
- [Ce qu'il ne remplace pas](#)
- [Environnement et cout](#)
- [Petite histoire](#)
- [Biais et angles morts](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Illusion de comprehension](#)
- [Hors distribution : trois exemples](#)
- [Limites legales et organisationnelles](#)

#### 19. [Chapitre 19 - Bonnes pratiques deep learning](#)

- Ce que ce n'est pas
- Avant
- Pendant
- Après
- Usage LLM couple
- Petite histoire
- Reproductibilité minimale
- Erreur classique
- En vrai
- A toi
- Culture de communication
- Les 12 cases (modele)
- Séparer jouet et production

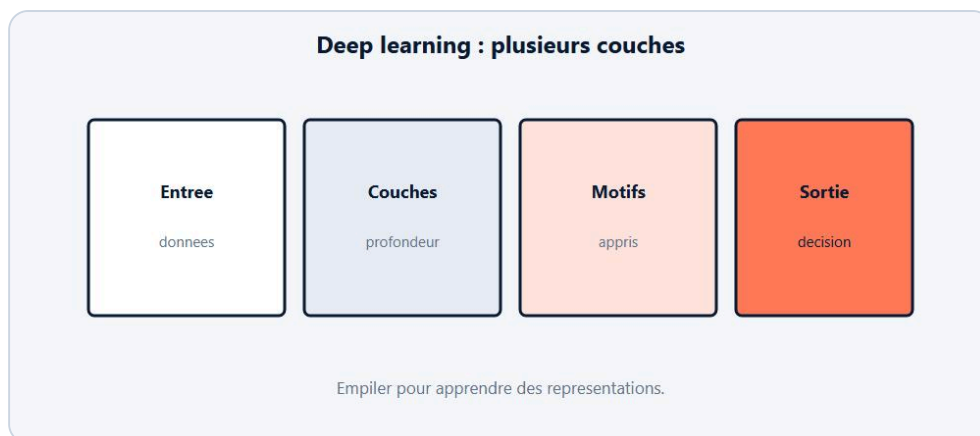
## 20. Chapitre 20 - Quiz final

- Avant de commencer
- Questions
- Petite histoire de correction
- Corriges
- Bonus oral
- En vrai
- A toi
- Pourquoi ces distracteurs
- Après le score

## 21. Chapitre 21 - Bravo

- Ce que tu emportes
- Petite histoire
- La suite possible
- Erreur classique après un livre
- En vrai
- A toi
- Remerciement utile
- Message à toi dans trois mois
- Partager sans jargon

## Chapitre 1 - Salut, c'est quoi le deep learning ?



Deep learning : des reseaux a plusieurs couches.

Le **deep learning**, c'est une famille de machine learning qui utilise des **reseaux de neurones artificiels** a plusieurs couches. "Deep" veut dire profond : entre l'entree (pixels, sons, tokens de texte) et la sortie (classe, nombre, prochain mot), il y a beaucoup d'etages de transformation. Ces modeles ont casse des plafonds sur l'image, la voix, puis le langage. Les **LLM** dont on parle tous les jours sont, sous le capot, des architectures de deep learning specialisees - souvent des **transformers**.

Chez DanielCraft, on ne commence pas par les equations. On commence par l'intuition : empiler des filtres qui apprennent a reconnaitre des motifs de plus en plus abstraits. Sur une photo, ca peut aller des bords aux formes, puis aux objets. Sur du texte, des fragments aux structures, puis a des intentions probables. Tu n'as pas besoin de reinventer un laboratoire. Tu as besoin d'une carte mentale solide pour dialoguer avec des outils, des prestataires, et plus tard du code.

### ★ A RETENIR

Deep learning = reseaux a plusieurs couches qui apprennent aussi des representations, pas seulement une decision finale.

### Ce que ce n'est pas

Ce n'est pas une conscience. Ce n'est pas "toujours mieux que le ML classique". Sur un petit tableau de dix colonnes numeriques, un modele scikit-learn simple gagne souvent. Ce n'est pas non plus un cluster de GPU obligatoire pour comprendre : tu peux saisir l'essentiel avec du papier, des exemples, et un peu de pratique. Et ce n'est surtout pas une excuse pour deployer sans validation, sans metriques, sans plan si le modele se trompe.

Ce n'est pas non plus "un seul produit". CNN pour la vision, transformers pour le langage, modeles audio, pipelines mixtes : meme famille large, usages differents. On va les demeler sans jargon opaque.

Tu as une entree brute. Le reseau la transforme couche apres couche en une representation plus utile, puis en une decision. En ML classique, tu ciselais souvent les **features** a la main. En deep learning, les couches apprennent aussi des features internes. Cette automatisation a un prix : opacite, besoin de donnees et de calcul, risque d'**overfitting**. Le jeu n'en vaut la chandelle que si le probleme le demande - image, parole, texte long, motifs trop riches pour un tableur.



## ♦ ASTUCE

Note un probleme "image ou texte" ou le deep learning semble pertinent, et un probleme "petit tableau" ou un modele simple suffit. Cette distinction te suivra tout le livre.

## Fil rouge : Ines

Ines developpe une appli qui reconnait des pieces detachees sur photo. Elle veut aussi comprendre comment les assistants texte fonctionnent, parce que ses clients lui posent des questions et qu'elle refuse de repondre "c'est de la magie". Lea, freelance web, l'ecoute pour mieux briefier ses prestataires IA. Max, artisan, veut juste savoir pourquoi "l'IA photo" se trompe sur un chantier mal eclaire. Sam, enseignant, cherche des analogies claires pour ses eleves. Quatre regards, une meme carte.

## Ce que tu vas savoir faire

Dans ce livre : neurone, couches, activations, idee simple de la **backprop**, **CNN**, **RNN** (overview), transformers, overfitting en deep learning, idee des **GPU**, **transfer learning**, lien avec les LLM, ateliers, choix d'architecture, limites, bonnes pratiques, quiz. Tu sortiras capable d'expliquer a un ami comment ca marche "en gros", et de ne pas te faire impressionner par un jargon vide.

Niveau debutant solide. Pas besoin de coder pour comprendre le debut. Besoin de curiosite et d'honnetete : le deep learning aide ; il ne remplace pas ta verification ni ton jugement metier.



Exemple : l'entree traverse des couches jusqu'a une decision.

## Petite histoire

Ines avait 180 photos de pieces, prises au telephone sous un neon jaune. Un collegue lui a dit "entraîne un gros reseau from scratch, c'est du deep learning". Elle a perdu une semaine, obtenu 98 % sur le train et 41 % sur des photos nouvelles. Puis elle a bascule vers un modele preentraine, quelques couches adaptees, et une validation sur le vrai terrain. Le score a baisse sur le train. Le produit a commence a servir. Chez DanielCraft, on celebre ce genre de retournement : moins de theatre, plus de generalisation.

## Erreur classique

Croire que deep learning = toujours mieux. Ou croire qu'il faut un cluster de GPU pour comprendre. Autre piège : confondre "on a mis du deep learning" avec "le problème est résolu". Quand tu entends cette phrase, pose les questions du livre : combien de données ? préentraîne ou from scratch ? quelle validation ? quel budget calcul ? quel comportement hors distribution ? quel plan si le modèle se trompe ?

### ⚠ ATTENTION

"Deep" n'est pas un concours de profondeur. C'est un compromis entre signal, données, calcul et régularisation.

## En vrai

Prends un problème de ton monde. Ecris en deux colonnes : "deep learning possible ?" et "modèle simple suffit ?". Si tu hésites, c'est déjà un progrès : tu ne colles plus l'étiquette IA partout.

## A toi

Ecris un problème image ou texte où le deep learning semble pertinent, et un problème petit tableau où un modèle simple suffit. Une phrase de justification pour chaque. Garde ce papier : les ateliers y reviendront.

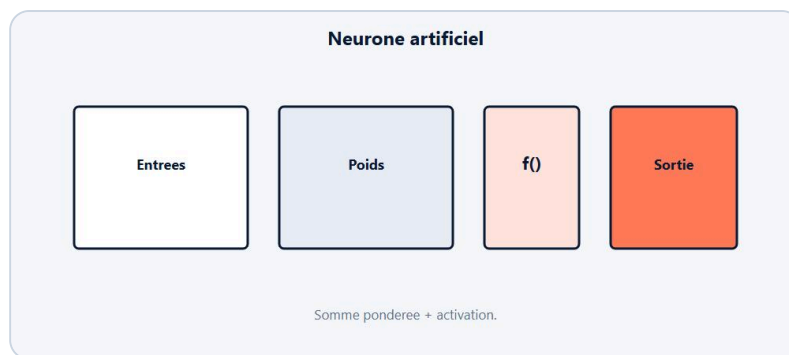
## Deep learning vs ML classique

Sur un CSV métier propre, le ML classique reste souvent le bon premier réflexe. Sur une image brute (des milliers de pixels) ou du texte long, le deep learning brille parce qu'il apprend aussi des représentations. En 2026, les deux coexistent dans les mêmes produits. Choisir, c'est matcher le problème, pas suivre la mode.

## Ce que "50 pages" doivent changer chez toi

A la fin, tu ne seras pas chercheur. Tu seras quelqu'un qui sait ce qu'est un neurone, pourquoi on active non linéairement, comment l'erreur remonte pour ajuster les poids, pourquoi un CNN aime l'image, pourquoi un transformer aime le contexte, pourquoi l'overfitting frappe fort, pourquoi le GPU accélère, pourquoi le transfer learning est le geste 2026, et comment un chat LLM se branche sur tout ça. C'est déjà beaucoup. C'est surtout actionnable demain matin.

## Chapitre 2 - Un neurone : une petite decision ponderee



Neurone artificiel : entrees, poids, sortie.

Un **neurone artificiel**, en version poche, prend des entrees numeriques, les multiplie par des **poids**, ajoute un **biais**, puis passe le resultat dans une fonction d'**activation**. Pense a une recette qui melange des ingredients avec des dosages, puis decide si le signal continue. Ce n'est pas une cellule vivante. C'est une petite fonction flexible, ajustable sur des exemples.

Chez DanielCraft, on insiste sur cette sobriete : si tu comprends un neurone, tu comprends le brique de base. Le reste du livre n'est "que" de l'organisation - couches, specialisations, astuces d'apprentissage - autour de cette brique.

### ★ A RETENIR

Neurone = entrees x poids + biais, puis activation. Les poids s'apprennent sur des exemples.

### Ce que ce n'est pas

Ce n'est pas une copie fidele du cerveau. Le nom s'inspire de la biologie ; l'ingenierie a pris ce qu'il lui fallait et a laisse le reste. Ce n'est pas non plus une unites qui "contient" a elle seule le concept chat, piece, ou spam. Les concepts emergent souvent de patterns distribues dans beaucoup d'unites. Et ce n'est pas une excuse pour fixer les poids a la main "logiquement" sur des taches complexes : l'apprentissage le fait mieux, a condition d'avoir des donnees.

Imagine predire "prendre un parapluie". Entrees possibles : probabilite de pluie, force du vent, distance a parcourir. Chaque entree a un poids. Si la pluie compte beaucoup, son poids grossit en valeur absolue. Si le vent est du bruit, il s'attenuie. Le biais decale le seuil de decision. L'activation decide comment ce score brut devient un signal utile - coupe, ecrase, laisse passer.

```
import numpy as np

x = np.array([0.8, 0.2, 0.5]) # pluie, vent, distance (normalisees)
w = np.array([2.0, -0.3, 0.4]) # poids appris (exemple)
b = -0.5

z = float(np.dot(w, x) + b)

def relu(t):
    return max(0.0, t)

y = relu(z)
print("score brut z =", round(z, 3), "| apres ReLU =", round(y, 3))
```

Change les poids a la main : tu sens deja qu'augmenter le poids "pluie" pousse la sortie. L'apprentissage fera ca automatiquement sur des exemples.

Ines, sur ses pieces, imagine des entrees plus abstraites plus tard (motifs de texture, contours). Mais le geste mental reste le meme : melanger, ponderer, activer.

## Pourquoi des poids ?

Les poids s'ajustent pendant l'apprentissage. Tu n'ecris pas la regle complete a la main ; tu laisses les exemples pousser les dosages. Chaque poids est un **parametre**. Plus tu en as dans un reseau, plus tu peux coller a des motifs complexes - et au bruit. La capacite n'est pas un trophée. C'est un budget a depenser avec des donnees et de la regularisation. Un neurone unique a peu de capacite ; un LLM en a une immense.

### ♦ ASTUCE

Quand quelqu'un dit "le modele a appris", traduis : "des poids ont bouge pour reduire une erreur mesuree".

## Limite d'un seul neurone

Un neurone seul est faible : il trace des frontieres simples. La puissance vient des **reseaux** : beaucoup de neurones organises en couches, qui composent des decisions. Comme une equipe : chacun voit un aspect ; ensemble ils resolvent un motif complexe. C'est pour ca que le chapitre suivant parle d'empiler.

## Petite histoire

Lea devait expliquer a un client pourquoi "un neurone" n'allait pas reconnaitre une piece metallique sur photo. Elle a dessine trois fleches (entrees), des dosages, une petite case "activation", une sortie. Le client a sourit : "donc c'est une balance ?". Presque. Une balance qui s'auto-regle sur des milliers d'exemples. Max, le meme jour, a compare ca a son "feeling" de chantier : il pondere humidite, temperature, type de joint. Difference : lui assume ; le neurone optimise un critere sans comprendre le metier.

## Erreur classique

Imaginer un neurone unique qui "contient" le concept. Autre piege : croire que plus de parametres = toujours plus intelligent. Sans donnees et sans controle, plus de parametres = plus de memorisation du train. Sam le dit a ses eleves : capacite sans discipline, c'est du par coeur.

### ⚠ ATTENTION

Le nom "neurone" est une metaphore utile. N'en deduis pas que le reseau "pense" ou "sent".

## En vrai

Prends une decision binaire de ton quotidien (sortir le linge, accepter un devis, classer un mail). Liste trois signaux d'entree. Imagine des poids. Tu viens de faire un mini-neurone mental - sans code.

## A toi

Invente 3 entrees pour predire "prendre un parapluie" (meteo, vent, distance). Imagine des poids. Ou ton intuition est-elle deja un mini-neurone ? Ecris-le en cinq lignes.

## Parametres et capacite

Chaque connexion poids compte. Dans un reseau profond, on parle de millions, parfois de milliards de parametres. Cette capacite explique les succes spectaculaires - et les echecs d'overfitting. Plus loin, on verra comment la **backprop** ajuste ces dosages, et comment le transfer learning reutilise des poids deja utiles plutot que de tout reinventer.

## Scene Ines

Ines pose sur une feuille : "entree brute = pixels", "sortie = classe de piece". Entre les deux, elle dessine des neurones en couches sans encore les detailer. Elle sait deja une chose : la decision finale n'est qu'une combinaison ponderee de signaux appris. Cette phrase lui evite de croire au mystere. Chez DanielCraft, c'est exactement le niveau de clarte qu'on vise au chapitre 2.

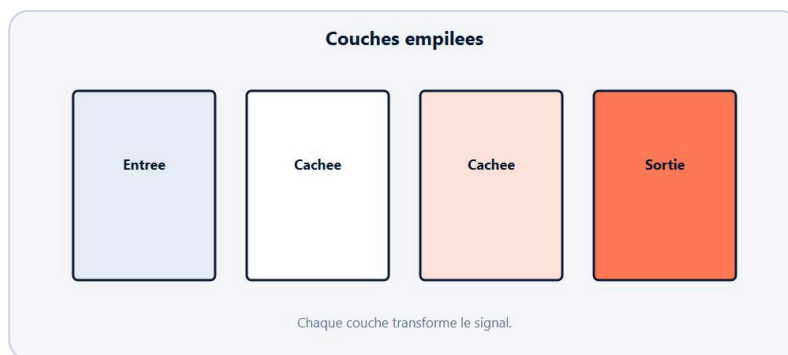
## Trois nombres pour sentir

Prends pluie=0.8, vent=0.2, distance=0.5. Imagine poids 2.0, 0.1, 0.5 et biais -1.0. Score brut =  $0.82 + 0.20.1 + 0.5*0.5 - 1.0 = 0.92$ . Avec ReLU, la sortie reste 0.92. Change la pluie a 0.1 : le score bascule. Tu viens de voir un neurone reagir sans framework. Chez DanielCraft, ce mini calcul vaut mieux qu'une slide animee : tu sens le dosage.

## Du neurone au reseau (teaser)

Demain, tu empileras. Garde cette idee : chaque neurone ne "voit" qu'une combinaison. La richesse nait de l'organisation. Ines ecrit sur son carnet : "1 neurone = 1 vote pondere ; le comite decide plus loin". C'est assez pour passer au chapitre couches sans mystere.

## Chapitre 3 - Couches : empiler des transformations



Couches : entree, cachees, sortie.

Une **couche**, c'est un groupe de neurones qui reçoivent les sorties de la couche précédente. Couche d'**entree** : tes données (pixels, embeddings de tokens, mesures). Couches **cachees** : transformations internes. Couche de **sortie** : prédiction - classes, nombres, tokens suivants. Empiler, c'est composer : chaque étage prépare le terrain pour le suivant.

Chez DanielCraft, on compare ça à une chaîne de fabrication. La première station nettoie et détecte des motifs simples. Les suivantes assemblent. La dernière étiquette. Si tu ajoutes des stations sans matière première ni contrôle qualité, tu n'obtiens qu'une usine chère qui mémorise le bruit.

### ★ A RETENIR

Une couche transforme une représentation en une autre. La profondeur est un compromis, pas un trophée.

### Ce que ce n'est pas

Ce n'est pas un concours : "plus deep = plus moderne = mieux". Trop de couches sans données, c'est de l'overfitting et du coût. Ce n'est pas non plus obligatoire de visualiser chaque neurone. Et ce n'est pas la même chose d'élargir une couche (plus de neurones) et d'approfondir (plus de couches) : les effets diffèrent.

Au début du réseau, l'entrée est brute. Au milieu, des motifs. À la fin, une décision. Une bonne représentation rend le problème plus simple pour la couche suivante. Sur une image, les premières couches d'un CNN tendent à détecter des motifs locaux simples ; plus loin, des formes plus abstraites. Dans le texte, des couches successives mélangent le contexte. Tu retiens l'idée : le réseau apprend aussi **comment représenter** l'entrée, pas seulement la dernière case à cocher.

Ines dessine trois étages pour ses pièces : entrée image, cachée "motifs", sortie "classe". C'est grossier. C'est déjà utile pour briefer.

```
# Empilement dense minimal (PyTorch) : entree -> cachee -> sortie
import torch
import torch.nn as nn

class MiniMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
```

```

nn.Linear(4, 8), # 4 mesures en entree
nn.ReLU(),
nn.Linear(8, 3), # 3 classes en sortie (scores bruts)
)

def forward(self, x):
    return self.net(x)

modele = MiniMLP()
x = torch.randn(2, 4) # 2 exemples, 4 features
logits = modele(x)
print(logits.shape) # torch.Size([2, 3])

```

Tu n'as pas encore appris : tu as juste empilé. L'apprentissage viendra avec une loss et des poids qui bougent.

## Profondeur et largeur

Plus de couches peuvent représenter des fonctions plus riches - mais coûtent plus cher à entraîner, et overfitent plus facilement si les données manquent. Élargir augmente la capacité locale ; approfondir compose des transformations. Les architectures modernes jouent sur des blocs répétés, des connexions résiduelles, des normes - détails que tu rencontreras plus tard. Ici, retiens le levier : **forme du réseau = hypothèse sur la structure du problème**.

### ♦ ASTUCE

Avant d'ajouter une couche, écris ce que tu espères qu'elle apporte. Si tu ne sais pas, ne l'ajoute pas encore.

## Fully connected vs spécialisé

Une couche **dense** (fully connected) relie tout à tout. Sur une grande image aplatie, les paramètres explosent. Les **CNN** partagent des filtres locaux. Les **transformers** utilisent l'attention pour relier des positions d'une séquence. L'architecture encode un a priori : localité pour l'image, dépendances contextuelles pour le langage. Choisir une famille, c'est choisir un a priori - on y revient au chapitre "choix d'architecture".

## Petite histoire

Sam a demandé à sa classe de dessiner un réseau spam/texte court : entrée, une cachée, sortie. Un élève a ajouté quinze couches "parce que deep". Sam a demandé : "tu as combien d'exemples labels ?" L'élève : "quarante". La classe a compris plus vite qu'avec un cours d'une heure. Lea, elle, a vu le même piège chez un prestataire qui vendait "un réseau à 40 couches" pour classer trois types de formulaires PDF. Elle a exigé une baseline simple d'abord. La baseline a presque suffi.

## Erreur classique

Ajouter des couches parce que "deep = moderne" sans mesurer. Ou ignorer la taille d'entrée : aplatir une image 4K dans une couche dense naïve, c'est une explosion de paramètres et souvent une mauvaise idée. Autre piège : croire que la couche de sortie "comprend" ; elle ne fait que lire la représentation fournie par le dessus.

### **⚠ ATTENTION**

L'architecture n'est pas une garantie de qualite. Donnees, validation et usage font le resultat.

### **En vrai**

Dessine sur papier un reseau a 3 etages pour une tache simple de ton monde. Nomme entree, cachee, sortie. Une phrase par etage : "ici j'espere que..."

### **A toi**

Dessine un reseau a 3 etages pour classer spam / texte court : entree, cachee, sortie. Que sort la derniere couche (2 scores ? 1 probabillite ?) ? Justifie en trois phrases.

### **Representations utiles**

Une representation utile separe mieux les classes, ou prepare mieux la generation du prochain token. Transfer learning = reutiliser un milieu deja riche. RAG cote LLM = injecter des faits dans le contexte plutot que de tout stocker dans les poids. Prompting = conditionner la representation de sortie sans maj de poids. Ces leviers reviendront ; ils parlent tous le langage des couches.

### **Scene Ines**

Ines compare deux plans : un reseau dense naif sur pixels aplatis, et un CNN preentraine. Le premier a l'air "simple a expliquer". Le second respecte la localite de l'image. Elle choisit le second, non par snobisme, mais parce que l'a priori matche le probleme. Chez DanielCraft, matcher le probleme bat impresionner la slide.

### **Lire une architecture comme un brief**

Quand un prestataire dit "on a 12 couches", demande ce que font les blocs, quelle entree, quelle sortie, quel preentrainement. Ines a appris a exiger un schema une page. Lea refuse les slides sans dimensions d'entree. Sam fait dessiner le schema avant tout jargon. La couche n'est pas un badge : c'est une hypothese sur la transformation.

### **Connexions residuelles (aperçu)**

Les reseaux tres profonds ajoutent parfois des raccourcis pour laisser le signal et le gradient circuler plus facilement. Tu n'as pas a les reinventer. Sache qu'elles existent, et que "plus profond" a ete rendu praticable aussi par ce genre d'astuces d'architecture, pas seulement par plus de GPU.



## Chapitre 4 - Activations : introduire du non-linéaire



Activation : tordre le signal pour du non-linéaire.

Si tu n'empilais que des additions et multiplications linéaires, plusieurs couches denses s'effondreraient en une seule transformation linéaire. Les fonctions d'**activation** cassent cette linéarité. Elles permettent au réseau de tordre l'espace et d'approximer des motifs complexes. Sans elles, la "profondeur" serait surtout du théâtre.

Chez DanielCraft, on retient une phrase : l'activation est le geste qui rend l'empilement utile. Tu n'as pas à collectionner vingt variantes. Tu as à comprendre le rôle, quelques classiques, et le piège de la saturation.

### ★ A RETENIR

Sans non-linéarité entre les couches, pas de profondeur utile. Activation = tordre le signal pour approximer du complexe.

### Ce que ce n'est pas

Ce n'est pas la même chose que la fonction de **perte** (loss). L'activation transforme des signaux à l'intérieur du réseau ; la loss mesure l'erreur pour apprendre. Ce n'est pas non plus une décoration de graphique. Et ce n'est pas "une seule activation pour tout" : les couches cachées et la sortie ont souvent des besoins différents.

Imagine un score brut qui sort d'une somme pondérée. **ReLU** laisse passer les positifs et coupe les négatifs : simple, souvent efficace en couches cachées. **Sigmoid** écrase vers 0..1 : utile parfois pour une probabilité binaire. **Softmax** transforme plusieurs scores en une distribution qui somme à 1 : langage courant des classificateurs multi-classes. Pour un LLM, la sortie ressemble à une distribution sur le vocabulaire : "quel prochain token est probable ?".

```
import numpy as np

def relu(z):
    return np.maximum(0, z)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def softmax(z):
    e = np.exp(z - np.max(z)) # stabilite numerique
    return e / e.sum()

z = np.array([-1.0, 0.5, 2.0])
print("ReLU   ", relu(z))
print("sigmoid", np.round(sigmoid(z), 3))
print("softmax", np.round(softmax(z), 3), "somme =", round(softmax(z).sum(), 3))
```

Ines, pour ses pieces, pense "scores bruts puis softmax vers classes". Elle n'a pas besoin de deriver la formule pour choisir la bonne idee.

## En sortie

La derniere activation depend de la tache : nombre libre (regression), probabilite (classification binaire), distribution sur classes (softmax), ou distribution sur vocabulaire (LLM). Choisir une mauvaise sortie, c'est comme demander a un thermometre d'afficher une couleur sans legende. Lea a vu un prestataire sortir des scores non normalises et les presenter comme des "pourcentages de confiance" a un client. Confusion garantie.

### ♦ ASTUCE

Avant de parler "confiance", demande : cette sortie est-elle une probabilite calibre, un score brut, ou une impression marketing ?

## Softmax en sortie (intuition)

Pour plusieurs classes, on obtient des scores bruts puis un softmax les transforme en nombres positifs qui somment a 1. Utile pour choisir la classe ou pour lire une incertitude relative. Ce n'est pas une garantie de calibration parfaite : un modele peut etre tres "sur" et faux. Sam le dit aux eleves : distribution n'egal pas verite.

## Petite histoire

Max a demande a Ines pourquoi "couper les negatifs" aidait. Elle a pris l'image d'un robinet : parfois tu veux eteindre un signal inutile pour laisser d'autres chemins s'exprimer. ReLU est brutal et efficace souvent. D'autres activations existent quand ReLU "tue" trop de neurones (dying ReLU) - variantes que tu rencontreras si tu codes. Pour comprendre, retiens le besoin de non-linearite d'abord.

## Erreur classique

Utiliser une activation qui sature trop et "tue" le **gradient** : l'apprentissage n'avance plus. D'ou la popularite de ReLU et variantes dans beaucoup de couches cachees. Autre piege : confondre activation et loss. Troisieme piege : lire un softmax comme une certitude metier sans evaluation hors distribution.

### ⚠ ATTENTION

Fluide et "99 %" a l'ecran n'egalent pas competent. Verifie sur des cas reels.

## En vrai

Explique a voix haute, en cinq lignes, pourquoi on active non lineairement entre les couches. Si tu bloques, relis l'image de l'effondrement lineaire.

## A toi

Explique a un ami (ou sur papier) pourquoi on active non lineairement entre les couches. Ajoute un exemple de sortie pour une tache binaire et une tache multi-classes.

## Lien avec la backprop

Au chapitre suivant, tu verras que l'apprentissage remonte une erreur a travers ces activations. Si une activation ecrase trop le signal, le message "monte / descends un peu" s'affaiblit. D'ou l'importance pratique du choix - pas pour le snobisme mathematique, pour que l'entrainement bouge.

## Scene DanielCraft

Lea prepare un atelier client. Elle ecrit au tableau : "lineaire empile = encore lineaire" puis "activation = pliure". La salle retient mieux que dix slides de formules. Chez DanielCraft, on prefere cette pliure mentale a une encyclopedie d'activations jamais utilisees.

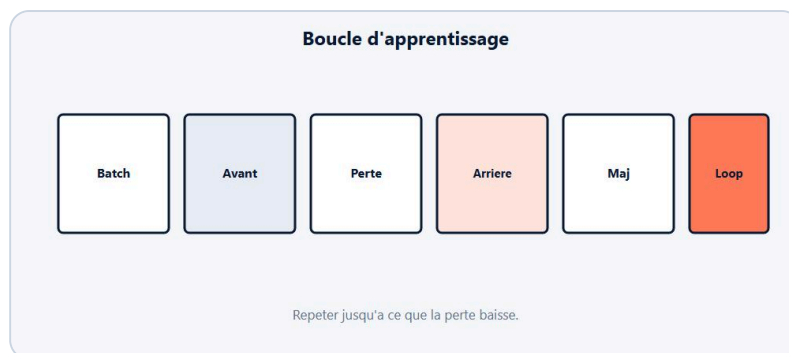
## ReLU vs saturations (terrain)

Ines a vu une fois une courbe plate : loss qui ne bouge presque plus. Cause probable : signaux ecrases, gradients faibles. Elle a change d'activation cachee vers une variante plus "passante", baisse legerement le learning rate, et la courbe a repris. Tu n'as pas a memoriser le nom de chaque variante. Tu as a reconnaitre le symptome : "plus rien n'apprend" peut venir d'une non-linearite mal choisie autant que d'un bug de labels.

## Activation et lecture metier

Quand un softmax sort 0.91 sur une piece, Lea demande : "calibre sur quel set ?". Si le set est studio et le terrain est neon, le 0.91 ment poliment. L'activation organise des nombres ; elle ne certifie pas le monde. Sam le fait repeter : distribution utile, verite a verifier.

## Chapitre 5 - Backpropagation : l'idée simple de l'apprentissage



Backprop : batch, avant, perte, arriere, maj, repeter.

La **backpropagation** (retour d'erreur), c'est la methode pour ajuster les poids apres avoir vu une erreur. Version histoire : le reseau fait une prediction (passe avant). On compare a la cible avec une fonction de **perte**. Puis on propage en arriere de petites indications : "toi, poids, monte un peu ; toi, descends". On repete sur beaucoup d'exemples, souvent par petits lots (**mini-batches**).

Chez DanielCraft, on veut que tu puisses raconter la boucle sans notes. Pas deriver sur un tableau avant le cafe. Raconter. Si tu racontes juste, tu ne te feras plus avoir par un jargon qui dit "on backprop" comme un sortilege.

### ★ A RETENIR

Apprendre = reduire une erreur mesuree en ajustant des poids via passes avant / arriere. La backprop calcule comment chaque poids a contribue a l'erreur.

### Ce que ce n'est pas

Ce n'est pas une comprehension du sens. La backprop optimise un critere. Si le critere est mal choisi, elle optimise mal - brillamment. Ce n'est pas non plus "comprendre comme un humain". Et ce n'est pas quelque chose que tu dois reprogrammer a la main en 2026 : les frameworks (PyTorch, TensorFlow...) calculent les gradients. Ton job : donnees, architecture raisonnable, loss adaptee, monitoring, regularisation.

Imagine etre dans le brouillard sur une colline et vouloir descendre : tu sens la pente sous tes pieds et tu fais un pas dans la direction qui descend. Le **gradient** est cette pente dans l'espace des poids. Le **taux d'apprentissage** (learning rate) est la taille du pas. Trop grand : tu sautes et tu diverges. Trop petit : tu rampes eternellement. Ines a vu une loss exploser apres avoir "mis le learning rate a fond pour aller plus vite". La colline n'aime pas les grands sauts aveugles.

```
# Une seule etape "apprendre" sur un neurone lineaire (MSE)
import numpy as np

x, y_vrai = 2.0, 5.0
w, b = 0.0, 0.0
lr = 0.1

for etape in range(8):
    y_pred = w * x + b
    loss = (y_pred - y_vrai) ** 2
    # gradients de la MSE
    dw = 2 * (y_pred - y_vrai) * x
    db = 2 * (y_pred - y_vrai)
    w -= lr * dw
    b -= lr * db
    print(f"etape {etape}: pred={y_pred:.2f} loss={loss:.3f} w={w:.3f} b={b:.3f}")
```

Tu vois la prediction se rapprocher de 5. C'est l'idee de la backprop, en tout petit, sans framework.

#### ♦ ASTUCE

Quand la loss train ne baisse pas, demande d'abord : learning rate, bugs de donnees, mauvaise loss - avant d'ajouter dix couches.

## Epochs et batches

Une **epoch** : un passage sur l'ensemble d'entrainement. Un **batch** : un sous-groupe d'exemples pour une mise a jour. Ces details changent stabilite et vitesse. En pratique, tu observes la courbe de loss train / validation. Si le train plonge et la validation remonte, tu n'as pas un succes : tu as souvent de l'overfitting en cours.

## Optimizers (aperçu)

SGD, Adam et cousins : des facons d'utiliser les gradients pour mettre a jour les poids, avec des astuces de moment et d'adaptation du pas. Tu n'as pas a les reinventer. Sache qu'ils existent, qu'ils ont des reglages, et que le learning rate reste souvent le levier le plus sensible. Lea note dans ses briefs prestataires : "quels optimizer et learning rate, et comment vous arretez l'entrainement ?"

## Petite histoire

Sam a fait jouer la boucle a voix haute : batch, avant, loss, arriere, maj des poids, repeter. Un eleve a dit "c'est comme corriger une recette apres chaque degustation". Presque : sauf que tu corriges des milliers de dosages a la fois, un peu, souvent. Max a ajoute : "et si ma degustation est biaisee, je corrige dans le mauvais sens". Exact. Labels sales = apprentissage confiant vers le faux.

## Erreur classique

Croire que backprop "comprend". Autre piege : juger un entrainement seulement sur la loss train. Troisieme piege : changer cinq hyperparametres a la fois et ne plus savoir ce qui a aide. Change une chose, logge, compare a une baseline.

### ⚠ ATTENTION

Une loss train qui baisse n'est pas un produit. La validation et le terrain decident.

## En vrai

Raconte la boucle en 6 etapes orales : batch, avant, loss, arriere, maj des poids, repeter. Chronometre-toi. Moins de 45 secondes nettes = socle acquis.

## A toi

Ecris les 6 etapes sans regarder. Puis ajoute une 7e ligne : "ce que je regarde pour savoir si ca generalise".

## Ce que tu retiens sans maths

Des millions (parfois milliards) de poids bougent pour reduire une erreur. Les frameworks font le calcul. Toi, tu choisis le probleme, tu soignes les donnees, tu lis les courbes, tu arrete quand la validation stagne (**early stopping**), tu refuses le theatre du "encore une epoch pour voir". Chez DanielCraft, l'apprentissage est une boucle disciplinaire autant qu'une formule.

## Scene Ines

Ines lance un petit entrainement de transfer learning. Elle regarde train et val. A l'epoch 7, train continue de baisser, val stagne. Elle stoppe, sauvegarde le meilleur checkpoint val, inspecte les erreurs. Elle n'a pas "fini le deep learning". Elle a pratique la backprop comme un pilote : capteurs, pas foi aveugle.

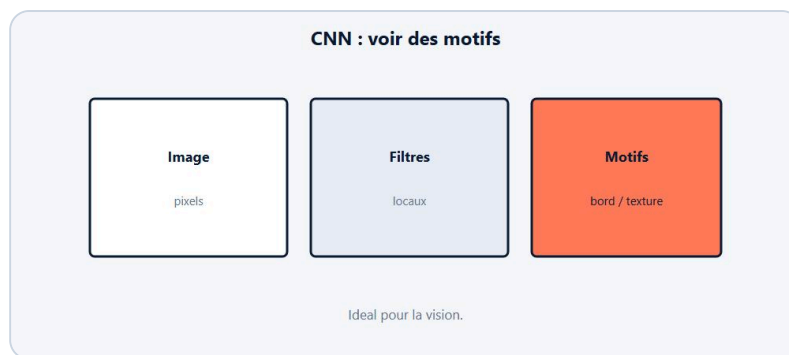
## Loss : le critere que tu choisis

Cross-entropy pour classer, erreur quadratique pour un nombre : la loss dit au reseau quoi minimiser. Si tu punis mal, tu obtiens un comportement confiant et hors sujet. Ines a un jour optimise l'accuracy globale alors que rater une piece critique coutait dix fois plus. Elle a change le critere (poids d'erreur, metrique metier). La backprop a alors pousse les poids dans une direction plus utile. Chez DanielCraft, on repete : optimiser juste un mauvais objectif, c'est encore se tromper.

## Mini-batch : pourquoi pas tout d'un coup

Tout le dataset a chaque pas serait trop lent et parfois trop lisse. Un exemple seul serait trop bruyant. Le mini-batch est le compromis. Tu n'as pas a choisir la taille parfaite jour 1. Tu dois savoir que ce levier existe, et qu'il change la stabilite des mises a jour.

## Chapitre 6 - CNN : voir par filtres locaux



CNN : voir des motifs dans les images.

**CNN** signifie Convolutional Neural Network : reseau a **convolutions**. Idee : au lieu de relier chaque pixel a chaque neurone (trop cher, trop de parametres), on fait glisser de petits **filtres** qui detectent des motifs locaux - bords, textures - partages sur toute l'image. Puis on empile : motifs plus riches, cartes de features, souvent reduction spatiale, puis decision.

Chez DanielCraft, c'est le chapitre vision par excellence. Si tu bosses l'image, tu dois sentir pourquoi la localite compte. Si tu ne bosses pas l'image, tu comprends quand meme pourquoi "aplatir une photo dans un dense" est souvent une mauvaise premiere idee.

### ★ A RETENIR

CNN = filtres locaux partages qui glissent sur l'image. A priori de localite + partage de poids.

### Ce que ce n'est pas

Ce n'est pas la seule architecture vision en 2026 (les vision transformers existent aussi). Ce n'est pas magique avec 200 photos from scratch. Ce n'est pas non plus "regarder" comme un humain : ce sont des motifs statistiques appris. Et ce n'est pas un permis de juger sur des photos studio pour deployer sur du flou de telephone.

Un chat peut etre a gauche ou a droite : un filtre d'"oreille" utile partout grace au partage de poids. Un pixel depend surtout de son voisinage : la localite est un a priori puissant. Les premieres couches tendent vers des motifs simples ; plus loin, des formes plus abstraites. Ines imagine deux filtres mentaux pour ses pieces : "contour metallique", "trou de vis". Elle sait que le reseau apprendra ses propres filtres ; l'exercice sert a sentir le besoin.

```
# CNN miniature (PyTorch) : image 1x28x28 -> 10 classes
import torch.nn as nn

cnn = nn.Sequential(
    nn.Conv2d(1, 8, kernel_size=3, padding=1), # filtres locaux
    nn.ReLU(),
    nn.MaxPool2d(2),                          # reduction spatiale
    nn.Flatten(),
    nn.Linear(8 * 14 * 14, 10),               # tete de classification
)
# Entree typique : batch x canaux x H x W, ex. (32, 1, 28, 28)
```

En vrai, Ines preferera souvent un backbone preentraîne + une tête fine-tunée plutôt que ce jouet from scratch.

#### ♦ ASTUCE

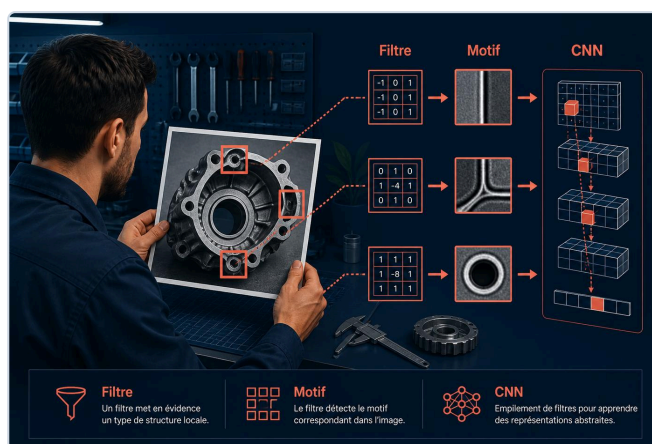
Avant d'entraîner, décris les conditions réelles : éclairage, flou, angle, téléphone. Le modèle vivra là-bas, pas dans ton dossier "jolies photos".

## Usages

Classification d'images, détection d'objets, segmentation, contrôle qualité sur chaîne, lecture de documents scannés (avec d'autres briques). Ines peut entraîner ou, plus souvent, réutiliser un modèle préentraîné (**transfer learning**) sur ses pièces détachées. Lea demande à ses prestataires vision : "préentraîné ou from scratch ?", "quelles augmentations ?", "quel set terrain ?".

## Data augmentation utile

Rotation légère, crop, flip si pertinent, variation de luminosité - pour apprendre l'invariance. N'augmente pas d'une façon qui change le label : un flip peut casser un symbole directionnel. Toujours se demander : cette transformation existe-t-elle dans le vrai monde de déploiement ? Max a ri : "retourner ma pièce à l'envers, ça arrive ; écrire du texte à l'envers sur mon compteur, non".



Exemple : filtres locaux sur une photo de pièce.

## Petite histoire

Ines a 180 photos. Tentation : CNN profond from scratch. Résultat : overfitting. Elle passe à un backbone préentraîné, remplace la tête de classification, augmente prudemment, valide sur photos chantier. Le score "labo" baisse. Le score "téléphone sous néon" monte. Sam utilise cette histoire en cours : la réalité bat le leaderboard interne.

## Erreur classique

Entraîner un CNN profond from scratch avec trop peu d'images. Préfère transfer learning + augmentation avec prudence. Autre piège : juger sur studio, déployer sur terrain. Troisième : data augmentation absurde qui crée des exemples impossibles et trompe le métier.



### **⚠ ATTENTION**

Un beau score sur un set propre ne prouve rien sur le flou, la pluie, ou l'angle bizarre du client.

### **En vrai**

Prends 10 photos de ton probleme (ou imagine-les). Note 3 variations que le modele verra forcement en production. Ton plan d'augmentation doit en couvrir au moins deux sans casser le label.

### **A toi**

Decris 2 filtres mentaux utiles pour ton probleme vision (ex. "contour metallique", "trou de vis"). Pourquoi la localite aide ? Cinq a huit lignes.

### **Lien avec la suite**

Le chapitre overfitting te dira comment lire les courbes. Le transfer learning te dira comment reutiliser un cerveau deja forme. L'atelier CNN te fera ecrire un plan de projet sans pretendre avoir deja 10 000 labels. Chez DanielCraft, le plan honnete vaut mieux que le notebook theatral.

### **Partage de poids : economie et invariances**

Un filtre de 3x3 sur une grande image reutilise les memes dosages partout. Moins de parametres qu'un dense naif. Meilleure chance d'apprendre un motif "bord" utile a gauche comme a droite. C'est pour ca que les CNN ont domine la vision pendant des annees. Les vision transformers changent parfois la donne aujourd'hui ; l'intuition convolution reste un socle. Lea demande quand meme : "pourquoi cette famille pour mon cas ?" - la reponse doit citer la localite, pas la mode.

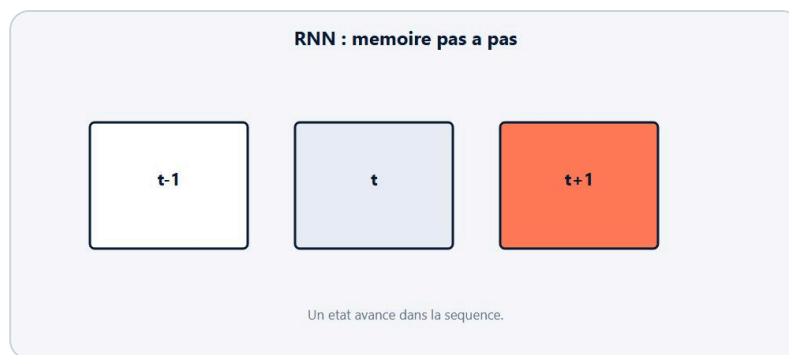
### **De la carte de features a la decision**

Apres plusieurs convolutions et reductions spatiales, tu obtiens des cartes plus abstraites, puis souvent une tete dense ou une tete de detection. Ines n'a pas besoin de dessiner chaque tenseur. Elle doit savoir ou brancher le transfer : souvent garder le corps, changer la tete, evaluer sur le terrain.

### **Detection et segmentation (aperçu)**

Classer toute l'image est un cas. Detecter des objets avec des boites, ou segmenter pixel par pixel, demande d'autres tetes et plus de labels. Ines commence souvent par la classification de piece entiere. Elle note detection comme etape 2 si le besoin metier le prouve. Ne saute pas les marches : chaque marche a son cout d'annotation.

## Chapitre 7 - RNN : sequences dans le temps (overview)



RNN : un etat memoire avance pas a pas.

Les **RNN** (Recurrent Neural Networks) ont ete concus pour les **sequences** : texte, audio, series temporelles. Idee : traiter un element a la fois en maintenant un **etat memoire** qui resume le passe. Chaque nouveau mot (ou mesure) met a jour la memoire, puis on predit. C'est l'intuition "le contexte voyage dans le temps".

Chez DanielCraft, on en parle en overview : utile pour l'histoire du NLP et pour certaines taches sequentielles, pas comme le reflexe 2026 par default sur le langage generatif massif. Les **transformers** ont souvent pris le dessus a grande echelle. Comprendre les RNN t'evite quand meme de croire que le texte n'est qu'un sac de mots.

### ★ A RETENIR

RNN = etat qui se met a jour a chaque pas de la sequence. Intuition pedagogique forte ; dominance transformers sur beaucoup de taches langage modernes.

### Ce que ce n'est pas

Ce n'est pas une comprehension humaine du recit. Ce n'est pas non plus "obsolete donc inutile a connaitre" : l'intuition reste, et des variantes / hybrides existent encore. Ce n'est pas le meilleur choix automatique en 2026 pour un chatbot ou un resume long : un modele preentraîne transformer resoudra souvent mieux, plus vite a l'usage.

Tu lis un journal de bord ligne apres ligne. Tu gardes en tete un resume. Chaque nouvelle ligne modifie le resume. A la fin, tu classes ou tu predicts la suite. Le RNN fait un geste voisin, en vecteurs. Sur les longues sequences, les RNN simples oublient ou deviennent instables : gradients qui disparaissent ou explosent. Des variantes (**LSTM**, **GRU**) ont ameliore la memoire. Puis l'attention des transformers a change la donne en permettant de relier des positions plus directement, et de mieux paralleliser sur GPU.

```
# RNN "jouet" : etat h mis a jour a chaque pas
import numpy as np

def tanh(z):
    return np.tanh(z)

h = np.zeros(4)          # memoire initiale
Wx = np.random.randn(4, 3) * 0.1
Wh = np.random.randn(4, 4) * 0.1
b = np.zeros(4)

sequence = [np.array([1, 0, 0]), np.array([0, 1, 0]), np.array([0, 0, 1])]
for t, x_t in enumerate(sequence):
    h = tanh(Wx @ x_t + Wh @ h + b)
    print(f"pas {t}: ||h||={np.linalg.norm(h):.3f}")
```

Tu sens le geste : a chaque pas, l'etat change. Sur du texte long en 2026, tu prefereras souvent un transformer preentraine - mais l'intuition "memoire qui voyage" reste utile.

#### ♦ ASTUCE

Quand tu entends "sequence", demande : qu'est-ce qui depend du passe recent ? du passe lointain ? du futur (attention au split temporel) ?

## Ou les situer en 2026

Pour le langage generatif massif : transformers. Pour comprendre l'histoire du NLP et certaines taches sequentielles legeres : utile. Pour l'audio et la parole, des architectures mixtes existent. Pour les series temporelles industrielles : parfois RNN/LSTM, parfois modeles statistiques, parfois transformers temporels. La lecon transversale : respecter le temps dans le split (pas de futur qui fuit dans le passe), et evaluer sur des periodes realistes.

## Petite histoire

Lea a herite d'un vieux notebook 2017 "RNN pour classer des tickets". Ca marchait mediocrement. Elle a remplace par un modele texte preentraine + une tete simple. Gain net, moins de maintenance. Max, lui, regarde des series de temperature de four : la, une approche sequentielle classique + regles metier reste pertinente. Sam insiste : choisir selon le probleme, pas selon la date du tutoriel.

## Erreur classique

Choisir un RNN parce qu'un tutoriel de 2017 le dit, alors qu'un modele preentraine transformer resoudrait mieux ton cas texte. Autre piege : croire qu'une memoire recurrente egale une comprehension. Troisieme : splitter une serie temporelle au hasard comme un CSV iid, et laisser le futur contaminer le passe.

#### ⚠ ATTENTION

Le temps n'est pas une feature comme une autre. Fuite temporelle = illusion de performance.

## En vrai

Donne un exemple de sequence dans ton metier : logs, phrases, mesures, messages clients. Entoure ce qui depend du passe recent. Une fleche suffit.

## A toi

Ecris ton exemple de sequence. Qu'est-ce qui depend du passe recent ? Du passe lointain ? Comment splitterais-tu train / validation dans le temps ?

## Series temporelles

Predire la demande, detecter une anomalie sur un capteur : le deep learning n'est pas obligatoire. Parfois un modele statistique simple + seuil metier gagne. Parfois un LSTM aide. Parfois un transformer temporel. Ines note dans son carnet : "sequence oui, deep learning peut-etre". Cette prudence est un acquis du livre.

## Pont vers les transformers

Le chapitre suivant remplace l'etat qui avance pas a pas par une idee d'**attention** : chaque position peut regarder les autres et ponderer. Tu garderas l'intuition sequence ; tu changeras surtout le mecanisme. Chez DanielCraft, on enseigne les RNN comme un pont, pas comme une destination forcee.

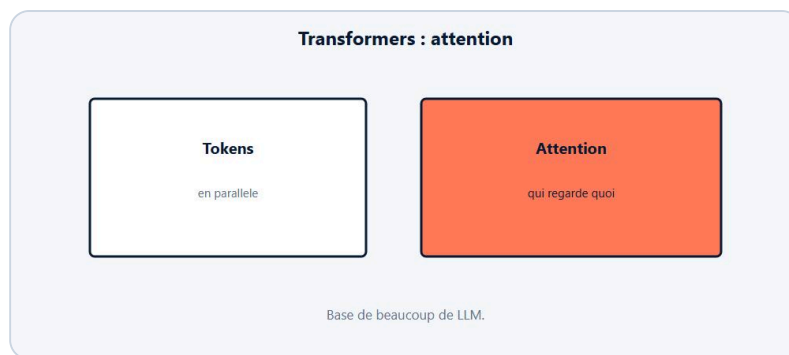
## LSTM / GRU en une image

Imagine une memoire avec des portes : quoi ecrire, quoi oublier, quoi lire. Les LSTM et GRU ont ajoute ce genre de controle pour mieux tenir sur des sequences plus longues que les RNN naifs. Tu n'as pas a coder les portes. Tu retiens : on a tente de soigner la memoire avant que l'attention des transformers ne propose un autre chemin, plus parallelisable.

## Split temporel : scene concrete

Max a des releves horaires. S'il melange hier et demain dans le train au hasard, le modele "voit" le futur. Score magique, deploiement mediocre. Regle : entrainer sur le passe, valider sur une periode plus recente non vue, tester plus tard encore. Chez DanielCraft, cette hygiene temporelle compte autant que le choix RNN vs autre.

## Chapitre 8 - Transformers : l'attention (overview)



Transformers : attention et contexte.

Le **transformer** est l'architecture dominante du NLP moderne et le coeur de beaucoup de **LLM**. Idee centrale : l'**attention**. Chaque position (chaque **token**) peut regarder les autres positions et ponderer celles qui comptent pour construire une nouvelle representation. Contrairement aux RNN classiques, on peut paralleliser beaucoup mieux sur **GPU**.

Chez DanielCraft, on veut une image nette, pas un papier de recherche. Si tu retiens "attention + profondeur + donnees + calcul", tu as le moteur. Le reste du produit (alignement, interface, garde-fous) vient autour.

### ★ A RETENIR

Transformer = attention entre positions / tokens, empilee en profondeur. Coeur de beaucoup de LLM.

### Ce que ce n'est pas

Ce n'est pas une garantie de qualite : "on a mis un transformer" n'egal pas "c'est bon". Ce n'est pas l'attention humaine consciente. Ce n'est pas magique hors donnees et evaluation. Et ce n'est pas gratuit : l'attention "tous sur tous" coute cher quand la sequence s'allonge - d'ou les limites de **contexte** et le prix des tokens.

Quand tu lis "le chat sur le tapis, il dort", le mot "il" doit se relier a "chat". L'attention apprend des liens utiles selon la tache. **Multi-tetes** : plusieurs types de liens en parallele. Empilement de blocs : representations de plus en plus riches. On ajoute des encodings de position, parce que sinon l'ordre serait moins clair. Ines explique ca a Lea avec un surligneur mental : chaque mot eclaire les autres selon le besoin.

```
# Attention "jouet" : 3 tokens, scores manuels puis softmax
import numpy as np

def softmax(z):
    e = np.exp(z - np.max(z))
    return e / e.sum()

# Tokens : [le, chat, dort] - scores d'attention du mot "dort" vers les autres
scores = np.array([0.2, 2.5, 0.1]) # "chat" attire fort
poids = softmax(scores)
print("poids attention:", np.round(poids, 3))
# Si chaque token a un vecteur valeur v_i, sortie = somme poids_i * v_i
```

Ce n'est pas le code d'un LLM. C'est le geste : ponderer les voisins selon l'utilite.

#### ♦ ASTUCE

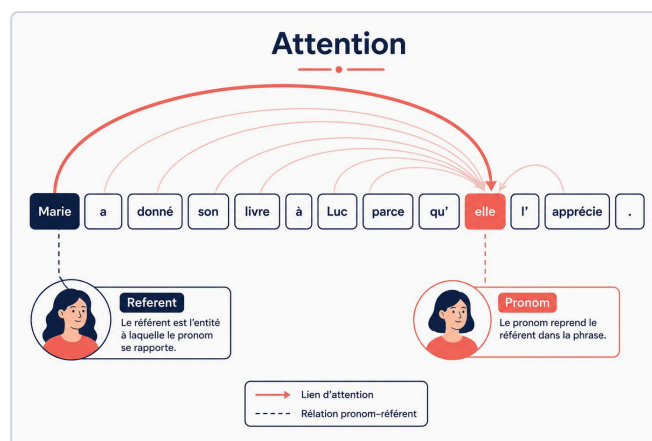
Pour sentir l'attention, prends une phrase avec un pronom. Demande : a quoi doit se coller ce pronom ? C'est le geste.

## Encodeurs, decodeurs, seq2seq

Selon les modeles : encodeur seul (classification, embeddings), decodeur seul (generation de texte style GPT), encodeur-decodeur (traduction...). Tu n'as pas a tout memoriser. Retiens : attention + profondeur + plein de donnees + plein de calcul = capacites emergentes de langage. Sam resume pour sa classe : "certains lisent et classent, d'autres ecrivent la suite, d'autres traduisent".

## Contexte et complexite

L'attention tous-sur-tous coute cher avec la longueur. D'où les fenetres de contexte limitees, les resumes, les decoupes, le **RAG** (chercher des morceaux puis les coller dans le contexte), ou le prix plus eleve des longues fenetres. Comprendre ca, c'est comprendre pourquoi un PDF de 200 pages ne "rentre" pas tel quel dans la tete du modele, meme si le chat a l'air confiant.



Exemple : l'attention relie un pronom a son referent.

## Petite histoire

Max a colle un devis de 40 pages dans un chat et a demande "resume les risques". Le modele a invente un delai. Lea lui a montre : fenetre limitee, attention diluee, hallucination possible. Ils ont coupe en sections, pose des questions cibles, verifie les chiffres. Meme architecture. Meilleur usage. Chez DanielCraft, l'architecture sans protocole reste un moteur sans frein.

## Lien pratique

**Fine-tuning**, RAG, **prompting** : facons d'utiliser ces geants sans tout reentrainer. Comprendre le transformer te permet de comprendre pourquoi le contexte a une taille limite, pourquoi les tokens coutent, pourquoi un long document doit parfois etre coupe ou cherche par morceaux. Ines, cote vision, note le parallele : representations riches reutilisees, tete adaptee - meme esprit que le transfer learning.

## Erreur classique

Dire "on a mis un transformer" comme garantie. Autre piege : confondre attention technique et attention humaine. Troisieme : noyer le modele sous un contexte inutile puis accuser "l'IA" d'etre bete.

### ⚠ ATTENTION

Le contexte est un budget. Remplis-le avec ce qui sert a la decision, pas avec tout le disque dur.

## En vrai

Explique l'attention a un ami avec l'exemple du pronom "il". Ajoute une phrase : pourquoi un GPU aide a entrainer ca (parallelisme des calculs de matrices / attention).

## A toi

Ecris ton explication "attention + pronom" en 8 lignes. Puis note une limite de contexte pour un document de ton metier : que coupes-tu en premier ?

## Du moteur au produit

Un LLM de chat, c'est souvent un transformer decodeur entraine a predire le prochain token, puis aligne pour mieux suivre des instructions. Le chapitre "lien LLM" detaille ce pont. Ici, tu as le coeur mecanique. Garde-le : il demystifie la suite.

## Multi-tetes : plusieurs regards

Une tete d'attention peut se specialiser vers la syntaxe, une autre vers des liens de long distance, une autre vers des motifs locaux - en pratique, tu n'inspectes pas chaque tete jour 1. L'idee utile : plusieurs types de liens en parallele enrichissent la representation avant la couche suivante. Sam compare a plusieurs eleves qui annotent la meme phrase avec des couleurs differentes, puis fusionnent.

## Pourquoi le GPU aime ca

Les calculs d'attention et de matrices se pretent au parallelisme. D'ou l'alliance historique transformers + accelerateurs. Comprendre ce lien t'evite de croire qu'un CPU portable entrainera un geant "parce que Python". Pour l'usage via API, le GPU est chez le fournisseur ; tu paies l'inference autrement. Ines distingue clairement les deux budgets.

## Chapitre 9 - Overfitting en deep learning



Overfitting : excellent sur le train, faible sur le neuf.

Les reseaux ont souvent des millions ou des milliards de **parametres**. Capacite enorme = facilite a coller au train. L'**overfitting** est donc central en deep learning, plus encore qu'avec un petit modele. Signes classiques : accuracy train haute, validation basse ; ecart qui se creusent au fil des epochs ; performances magiques sur un petit jeu trop vu.

Chez DanielCraft, on traite l'overfitting comme un risque de metier, pas comme une note de bas de page. Un modele qui recite le dossier d'entrainement et se plante sur le telephone du client n'est pas "presque bon". Il est dangereux s'il decide seul.

### ★ A RETENIR

Capacite sans donnees ni regularisation = memorisation. Surveillance train et validation ensemble.

### Ce que ce n'est pas

Ce n'est pas "le modele est trop intelligent". C'est souvent le contraire de la generalisation utile. Ce n'est pas non plus resolu en ajoutant toujours plus de couches. Parfois il faut mieux de donnees, pas plus de profondeur. Et ce n'est pas uniquement un probleme de vision : le texte, l'audio, les series temporelles overfitent aussi.

Imagine un eleve qui apprend par coeur dix sujets d'examen. Le jour J, le sujet change d'un cran : il panique. Le reseau fait un geste voisin s'il a trop de capacite et trop peu de diversite. La **generalisation**, c'est reussir sur des cas nouveaux issus du meme phenomene - pas reinventer le monde, mais ne pas coller au bruit du train.

Ines regarde deux courbes. Train plonge. Val remonte. Elle n'ajoute pas de neurones. Elle regularise, arrete tot, ou collecte mieux.



```
# Pseudo-boucle : early stopping quand val ne s'ameliore plus
meilleure_val = float("inf")
patience, sans_gain = 3, 0

for epoch in range(1, 21):
    loss_train = 1.0 / epoch          # fictif : train baisse
    loss_val = 0.4 + 0.05 * max(0, epoch - 8) # val remonte apres epoch 8
    if loss_val < meilleure_val - 1e-4:
        meilleure_val, sans_gain = loss_val, 0
        # sauvegarder les poids ici
    else:
        sans_gain += 1
    print(f"epoch {epoch}: train={loss_train:.3f} val={loss_val:.3f}")
    if sans_gain >= patience:
        print("early stopping a l'epoch", epoch)
        break
```

Le message : arreter, ce n'est pas abandonner. C'est proteger la generalisation.

## Remedes courants

Plus de donnees reelles. **Data augmentation** (surtout vision), avec prudence. **Dropout** (eteindre des neurones au hasard pendant l'entrainement). Weight decay / regularisation. **Early stopping** (arreter quand la validation n'ameliore plus). Architectures plus petites. **Transfer learning** plutot que from scratch. Label smoothing parfois. Et toujours : protocole de validation honnete, set de test rarement touche.

### ♦ ASTUCE

Liste tes remedes realistes "cette semaine" separement de tes remedes "un jour". L'action bat la wishlist.

## Courbes a surveiller

Loss train, loss val, metrique metier val. Si train plonge et val remonte : stop / regularise. Si les deux restent hauts : underfitting, ou donnees insuffisantes / mal alignees, ou bugs. Apprends a lire ces courbes comme un medecin lit une tension. Lea demande a ses prestataires des captures de courbes, pas seulement un score final.

## Generalisation dans le vrai monde

Meme avec une belle courbe, le deploiement peut casser si les photos changent (eclairage, telephone), si la langue change, si les utilisateurs detournent. Surveille. Recolte des cas d'echec. Reentraîne. Le deep learning n'annule pas la boucle ML : il l'intensifie. Max a vu un modele de "defaut visible" rate tous les defauts sous LED froide : le train etait sous LED chaude. Overfitting au contexte, pas seulement aux labels.

## Petite histoire

Sam a donne a sa classe un mini-jeu : 20 images, gros reseau, score train 100 %. Puis 5 images legerement differentes : catastrophe. Ensuite : modele plus petit + transfer + early stopping. Moins spectaculaire. Plus honnete. La classe a retenu. Chez DanielCraft, on garde cette pedagogie : faire sentir la claquette avant de vendre le remede.

## Erreur classique

Augmenter la taille du modèle dès que ça coince. Autre piège : data augmentation absurde qui change le label. Troisième : retoucher le test jusqu'à ce qu'il "marche" - tu overfitte alors au test lui-même.

### ⚠ ATTENTION

Ne touche pas au test final comme à un joystick. Sinon tu perds ton juge impartial.

## En vrai

Ouvre (ou imagine) une courbe train/val. Écris en une phrase ce que tu ferais à l'époque où val stagne. Si ta phrase est "encore 50 epochs", recommence.

## A toi

Liste 4 remèdes anti-overfitting applicables à ton idée de projet. Lesquels sont réalistes cette semaine ? Encadre-les.

## Capacité et humilité

Un petit modèle qui généralise un peu est parfois préférable à un grand modèle qui mémorise. En 2026, la tentation du "plus gros" est partout. Ton avantage, après ce chapitre, c'est de savoir dire non. Inès l'a écrit sur un post-it au-dessus de son écran : "val d'abord".

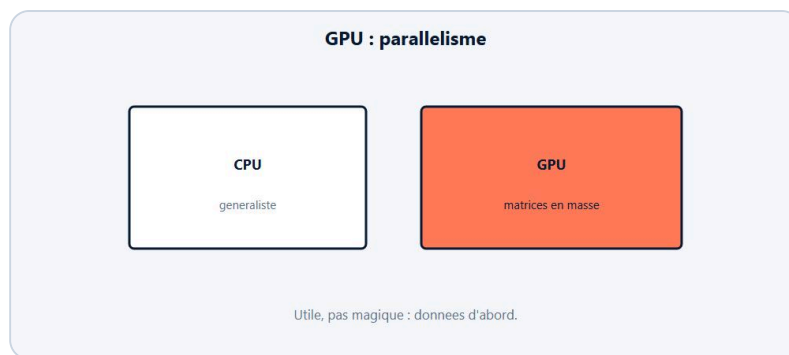
## Dropout et weight decay (intuition)

Dropout : pendant l'entraînement, tu "éteins" des neurones au hasard pour éviter que le réseau compte trop sur une co-adaptation fragile. Weight decay : tu pénalises des poids trop gros, ce qui freine la mémorisation excessive. Ce ne sont pas des sortilèges. Ce sont des freins. Léa demande lesquels sont actifs avant d'accepter un score. Early stopping reste souvent le frein le plus simple et le plus oublié.

## Overfitting au protocole

Tu peux aussi overfitte au test en le regardant trop souvent, ou à une augmentation absurde, ou à un seul téléphone. Élargis la notion : ce n'est pas seulement train vs val. C'est tout ce qui fait coller à un artefact au lieu du phénomène. Chez DanielCraft, on appelle ça "généraliser au vrai usage", pas au dossier du jour.

## Chapitre 10 - GPU : pourquoi ca va plus vite (idee)



GPU : parallelisme massif pour les matrices.

Un **GPU** (processeur graphique) est tres bon pour faire beaucoup de petites operations similaires en parallele - exactement le genre de calculs des reseaux (grosses multiplications de matrices, attentions, convolutions). D'ou son role central dans l'**entrainement** et parfois l'**inference** des modeles lourds.

Chez DanielCraft, on demystifie le fer. Tu n'as pas besoin d'acheter une carte pour comprendre. Tu as besoin de savoir pourquoi les tutos disent "il me faut un GPU", pourquoi l'inference LLM coute de l'argent, et comment choisir entre entrainer, fine-tuner, ou seulement appeler un modele existant.

### ★ A RETENIR

GPU = parallelisme massif pour des calculs de matrices. Utile ; pas un sesame moral ni un prerequis pour apprendre.

### Ce que ce n'est pas

Ce n'est pas obligatoire pour comprendre le deep learning. Ce n'est pas une garantie de bon modele. Ce n'est pas non plus "CPU = inutile" : le CPU reste excellent pour la logique variee, le data loading, beaucoup de pipelines. Et ce n'est pas un permis de louer du calcul cher avant d'avoir valide la question metier sur un sous-ensemble.

**CPU** : peu de coeurs tres flexibles, excellent pour des taches variees. **GPU** : des milliers de petits coeurs pour le throughput sur des operations similaires. Entrainer un gros CNN ou un transformer sur CPU seul peut etre lent jusqu'a l'absurde. Sur GPU (ou TPU / accelerateurs), ca devient praticable. Lea compare : "couteau suisse vs usine a parallele". Grossier. Utile.

### ♦ ASTUCE

Avant d'acheter ou louer, ecris : j'entraine, je fine-tune, ou j'inferre seulement ? Le besoin GPU change selon la reponse.

### Ce que ca change pour toi

Cloud, notebooks heberges, services geres, modeles deja entraines via API : autant de facons d'utiliser la puissance sans la posseder. Ines prototype d'abord avec un modele preentraine et un petit set. Elle ne loue du GPU serieusement qu'apres un go metier. Max, lui, n'a besoin que d'une API vision de temps en temps : zero carte chez lui, zero honte.

## Inference vs training

Entraîner est souvent le plus gourmand. Inferer (servir des predictions) peut aussi couter cher a grande echelle - millions de requetes LLM, latence, energie. Optimiser l'inference (quantization, batching, petit modele) est un metier. Pour un debutant : mesure avant d'acheter du fer. Sam fait estimer a ses eleves un cout mensuel grossier : requetes x tokens x prix, plus le temps de verification humaine.

## Limites memoire

La **VRAM** est finie. Un gros batch ou un long contexte peuvent saturer. Optimisations possibles plus tard : precision reduite, quantization, distillation, batching, gradient checkpointing. Idees, pas besoin de tout implementer jour 1. Retiens surtout : "out of memory" n'est pas une fatalite morale, c'est un signal de dimensionnement.

## Petite histoire

Un prestataire a propose a Lea "on loue huit GPU une semaine pour tout reentraîner". Lea a demande le baseline transfer learning sur un seul accélérateur léger. Resultat deja utile. Les huit GPU sont restes dans le devis, non signes. Chez DanielCraft, on aime cette scene : le calcul suit la preuve, pas l'inverse.

## Erreur classique

Croire que sans GPU perso on ne peut rien apprendre. Autre piege : louer du calcul cher sans valider la question metier. Troisieme : confondre "j'ai un GPU" et "mon protocole de validation est propre".

### ⚠ ATTENTION

Le fer accelere une boucle. Il ne remplace ni les labels, ni les metriques, ni le jugement.

## En vrai

Pour ton projet : entraîner, fine-tuner, ou inferer via un modele existant / une API ? Ecris la reponse en une phrase et le besoin GPU associe (aucun / ponctuel / serieux).

## A toi

Remplis le tableau mental : tache, volume de donnees, besoin calcul, alternative sans GPU. Une decision claire en bas de page.

## Energie et argent

Calcul = argent + energie. Utiliser le plus petit modele qui fait le job, reutiliser des poids preentraînes, preferer l'inference sobre : ce sont des gestes techniques et ethiques a la fois. Ines le note a cote de ses metriques. Ce n'est pas du greenwashing de slide ; c'est du dimensionnement adulte.

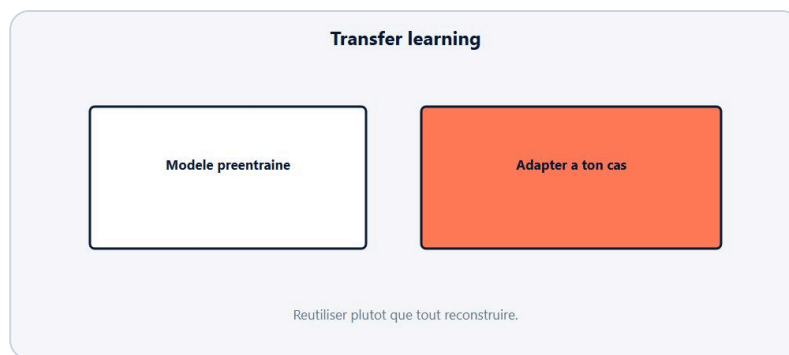
## Cloud, Colab, API : trois portes

Porte 1 : notebook heberge avec GPU ponctuel pour apprendre. Porte 2 : cloud loue a l'heure pour un fine-tune serieux. Porte 3 : API - tu n'as pas le fer, tu achetes l'inference. Ines utilise 1 pour prototyper, 3 pour certains textes, 2 seulement apres go metier. Max reste souvent en 3. Lea met les trois portes dans ses devis pour eviter le "il nous faut huit cartes" sans preuve.

## **Precision et quantization (aperçu)**

On peut parfois calculer avec moins de bits pour aller plus vite ou tenir en memoire. Quantization, distillation : mots que tu rencontreras si tu optimises l'inference. Jour 1, retiens seulement : il existe des leviers avant d'acheter plus de fer. Mesure d'abord le besoin.

## Chapitre 11 - Transfer learning : reutiliser un cerveau deja forme



Transfer learning : reutiliser un modele preentaine.

Le **transfer learning**, c'est repartir d'un modele deja entraine sur une tache large - reconnaitre des objets sur des millions d'images, ou un modele de langage sur d'énormes corpus - et l'adapter a ton cas avec moins de donnees. Au lieu d'apprendre tout **from scratch**, tu reutilises des representations utiles.

Chez DanielCraft, c'est souvent le geste 2026 par défaut. Les donnees labellisees coutent cher. Les modeles fondation existent. Reutiliser avant de reentrainer le monde n'est pas de la paresse : c'est de l'ingenierie.

### ★ A RETENIR

Transfer learning = reutiliser un modele preentaine et l'adapter. Moins de donnees, moins de calcul, souvent meilleur depart.

### Ce que ce n'est pas

Ce n'est pas une dispense de validation. Ce n'est pas magique si ton domaine est trop eloigne ou si tes labels sont sales - tu transfères aussi vers le sale. Ce n'est pas non plus "interdire le from scratch pour toujours" : parfois le domaine l'exige, avec budget et donnees. Et ce n'est pas synonyme exact de prompting : le prompting conditionne sans maj de poids ; le transfer, au sens strict, adapte des poids (même légèrement).

Tu embauches quelqu'un qui a déjà vu des millions d'objets, et tu lui apprends tes pieces detachees. Tu ne reprends pas l'alphabet visuel a zero. Ines telecharge un CNN preentaine, remplace la tete de classification, entraine surtout les dernieres couches. Sur le texte, on fine-tune légèrement, ou on fait du prompting / RAG sans tout retoucher. L'esprit est le même : partir d'un milieu déjà riche.

```
# Esquisse PyTorch : backbone gele + tete adaptee (N classes)
import torch.nn as nn
from torchvision import models

backbone = models.resnet18(weights="DEFAULT")
for p in backbone.parameters():
    p.requires_grad = False      # gele : on reutilise
backbone.fc = nn.Linear(backbone.fc.in_features, 5) # 5 classes metier
# Ensuite : entrainer surtout backbone.fc, learning rate petit, early stopping
```

#### ♦ ASTUCE

Ecris "ce que je reutilise" et "ce que j'adapte". Si les deux cases sont vides, tu n'as pas encore de plan transfer.

## Strategies

Geler beaucoup de couches, entrainer la tete. Puis parfois deverrouiller plus profond a petit learning rate. Adapter avec peu de parametres (**LoRA** et cousins dans le monde LLM). Surveiller l'overfitting : meme un modele preentaine peut coller a 100 images. Lea met dans ses contrats : "strategie de gel / degel documentee, learning rates, critere d'arret".

## Domaine shift

Si tes images de pieces sont grasses, floues, mal eclairees, et que le modele preentaine a vu des photos web propres, tu as un ecart de domaine. Le transfert aide encore souvent, mais prevois plus d'exemples cibles, une augmentation realiste, et une evaluation sur le vrai terrain. Max le rappelle : "mes photos ne sont pas Instagram".

## Petite histoire

Ines compare from scratch (echec) et transfer (debut utile). Elle gele le backbone, entraine la tete, regarde val. Puis elle degel quelques couches a petit pas. Elle documente. Quand un investisseur demande "vous avez invente l'architecture ?", elle repond "non, on a adapte proprement". Chez DanielCraft, cette honnetete vend mieux que le mythe du genie solitaire.

## Limites

Domaine trop eloigne : transfert plus difficile. Labels sales : transfert du sale. Licences et couts des modeles : a lire. Catastrophic forgetting : reentrainer tout a gros learning rate peut detruire les representations utiles. Sam fait un schema au tableau : "petit pas sur un geant > grand pas qui ecrase le geant".

## Erreur classique

Reentrainer tout a gros learning rate et detruire les representations utiles. Ou croire que transfer learning dispense de validation. Ou ignorer la licence du modele fondation.

#### ⚠ ATTENTION

Adapter n'abolit pas mesurer. Le preentaine arrive avec des biais et des angles morts : teste sur ton terrain.

## En vrai

Choisis un modele fondation (vision ou texte) que tu reuserais. Ecris ce que tu adapterais : classes, style, documents, tete de sortie.

## A toi

Decris le modele fondation, la strategie (gele / tete / LoRA...), le volume de donnees estime, et le critere go/no-go. Une demi-page max.

## Prompting, RAG, fine-tune

Trois leviers voisins, forces differentes. Prompting : rapide, pas de maj de poids. RAG : injecte des faits dans le contexte. Fine-tune / transfer : change le comportement du modele sur ton domaine. Ines les range dans cet ordre de cout. Elle monte d'un cran seulement si le cran d'en dessous ne suffit pas. C'est la discipline du chapitre.

## LoRA et adaptation legere (idee)

Dans le monde LLM, on adapte parfois avec peu de parametres ajoutes (LoRA et cousins) plutot que de tout retoucher. Esprit voisin du transfer : changer peu, garder beaucoup. Tu n'as pas a implementer LoRA ici. Tu dois savoir que "fine-tune" n'egal pas toujours "reecrire le geant". Lea exige qu'on precise la methode et le cout avant de signer.

## Licence et provenance

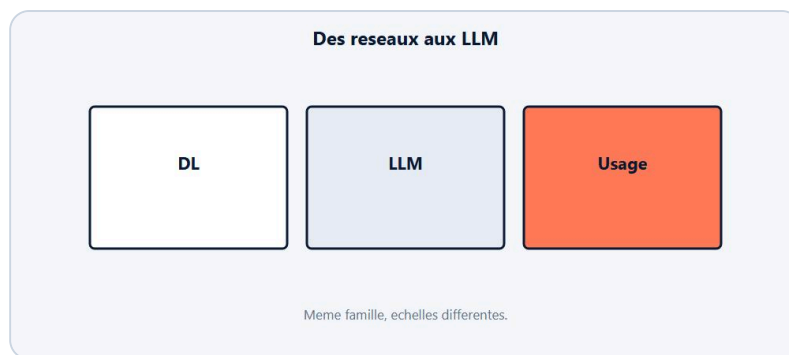
Un modele fondation a une licence, une provenance de donnees, parfois des restrictions commerciales. Lire avant d'integrer. Sam le met dans sa grille ethique scolaire version legere : "d'ou vient le cerveau que tu reuses ?". Chez DanielCraft, le transfer sans lecture de licence est une dette, pas une astuce.

## Ordre des leviers (rappel)

1) Modele preentraîne + tete. 2) Data meilleure / plus realiste. 3) Degeler un peu. 4) Methodes legeres type LoRA si LLM. 5) From scratch seulement avec preuves. Cet ordre evite 80 % des derivations couteuses. Lea l'imprime ; Max le simplifie en "reutiliser, mesurer, puis seulement compliquer".



## Chapitre 12 - Lien avec les LLM : du neurone au chat



Des reseaux profonds aux LLM.

Un **LLM** est un reseau de deep learning - souvent un **transformer** decodeur - entraine a predire le prochain **token** sur d'enormes corpus, puis souvent aligne pour mieux suivre des instructions et respecter des politiques. Quand tu chats, tu ne "parles pas a une conscience". Tu conditionnes une machine a suites de tokens.

Chez DanielCraft, ce chapitre ferme la boucle : tout ce que tu as vu (neurone, couches, activation, backprop, attention, GPU, transfer) se reconnaît dans l'outil du quotidien. Tu gagnes un vocabulaire anti-fantasme.

### ★ A RETENIR

LLM = deep learning langage (souvent transformer) + alignement + usage (prompt / RAG / agents). Tu pilotes ; tu ne dialogues pas avec une ame.

### Ce que ce n'est pas

Ce n'est pas sans lien avec le deep learning - au contraire. Ce n'est pas une base de connaissances fiable par default (**hallucinations**). Ce n'est pas un remplaceant automatique du ML classique sur un CSV metier. Et ce n'est pas "uploader un PDF une fois pour qu'il sache tout pour toujours" : sans RAG ni fine-tune evalue, le fichier du jour n'est pas grave dans les poids.

### Ce que tu reconnais maintenant

**Tokens** et **contexte** : la fenetre limite ce que l'attention peut croiser d'un coup. **Temperature** : reglage sur l'aleas des choix de tokens. Poids et couches : milliards de parametres ajustes par descente de gradient a l'echelle industrielle. Hallucinations : prediction plausible hors verite. Multimodal : on branche vision / audio sur des representations communes ou des modules couples. **RAG** : on injecte des extraits dans le contexte plutot que de tout memoriser dans les poids. **Agents** : on boucle le LLM avec des outils.

### ♦ ASTUCE

Dessine 8 fleches : donnees -> transformer -> tokens -> prompt -> reponse -> verification. Place RAG et temperature sur le schema.

## Entraîner vs utiliser

Entraîner un LLM fondation : hors de portée de la plupart des individus (données, GPU, argent, énergie). Utiliser : API, produit grand public, modèle open poids local selon machine. Fine-tuning léger : parfois accessible. Prompting et RAG : levier numéro un. Comprendre le deep learning te protège des fantasmes et t'oriente vers les bons leviers. Lea le dit à ses clients : "on n'entraîne pas ChatGPT ; on l'utilise proprement".

## Alignement (idée)

Après l'entraînement "prédire le prochain token", beaucoup de LLM passent par des étapes pour mieux suivre les instructions et réduire certains comportements dangereux. Ça n'efface pas les hallucinations. Ça change le tempérament du produit que tu utilises. D'où des différences de style entre outils. Sam compare : même famille de moteurs, permis de conduire différents.

## Petite histoire

Ines utilise un assistant pour rédiger la doc utilisateur de son appli vision. Elle fournit des faits vrais (classes, limites, seuils), interdit l'invention de chiffres, vérifie. Le LLM accélère le brouillon. Le CNN, lui, classe les pièces. Deux deep learning, deux rôles, une même posture de pilote. Max demande un mail client : même posture. Chez DanielCraft, on aime cette coexistence assumée.

## Ethique et coût

Derrière le chat fluide : calcul, énergie selon les infrastructures, travail d'annotation et de modération, risques de biais. Le chapitre n'est pas un procès. C'est une invitation à utiliser avec mesure, surtout en entreprise : données sensibles, vérification, abstention. L'argent des tokens n'est qu'une partie du coût : le coût d'une erreur confiante peut être bien plus haut.

## Erreur classique

Croire qu'un LLM rend inutile le ML classique sur un CSV. Autre piège : confondre "fine-tune" marketing et vrai changement de poids évalué proprement. Troisième : coller des secrets dans le fil "parce que ce sera plus perso".

### ⚠ ATTENTION

Fluide n'est pas compétent. Vérification et limites restent ton job.

## En vrai

Ouvre l'assistant auquel tu as accès. Pose une question de ton métier avec faits fournis, puis la même sans faits. Compare inventions et utilité. Note trois différences.

## A toi

Relie en schéma : données, transformer, tokens, prompt, réponse, vérification, RAG, température. Une flèche manquante = un trou à relire.

## **Pont vers les ateliers**

Tu vas solidifier l'intuition neurone/couches, écrire un plan CNN, puis relier transformer et usage LLM. Tu as maintenant le pont conceptuel. La suite fait faire. C'est le cœur pédagogique DanielCraft : comprendre, puis agir petit, puis évaluer.

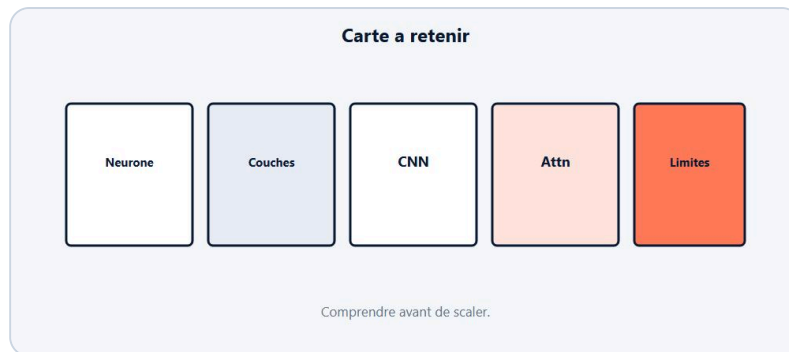
## **Temperature mentale**

Basse : plus déterministe, utile pour faits et procédures. Haute : plus variée, utile pour idées, risquée pour chiffres. Ines garde basse pour la doc technique. Lea monte un peu pour brainstorm, puis redescend pour l'envoi client. Le réglage n'est pas magique ; il change le profil de risque. Couple-le toujours à une vérification.

## **Agents : boucler avec freins**

Un agent enchaîne LLM + outils. Puissant, fragile. Sans freins (validation, plafonds, logs), tu automatises aussi les erreurs. Ce livre introductif dit : comprends le moteur avant d'ouvrir l'autoroute. Le chapitre bonnes pratiques et limites te rappellent le frein. Chez DanielCraft, l'agent vient après le protocole, pas avant.

## Chapitre 13 - A retenir



Carte DL : des neurones aux LLM.

Tu as une carte. Pas un diplôme de chercheur. Une carte. Le deep learning, ce sont des **reseaux a plusieurs couches** qui apprennent aussi des representations. Un **neurone** melange des entrees avec des poids, ajoute un biais, active. Les **couches** empilent des transformations. L'**activation** introduit le non-lineaire sans lequel la profondeur s'effondre. La **backprop** ajuste les poids a partir d'une erreur mesuree. Les **CNN** voient par filtres locaux. Les **RNN** portent une memoire sequentielle (overview utile). Les **transformers** misent sur l'attention, coeur de beaucoup de **LLM**. L'**overfitting** menace des que la capacite depasse le signal. Le **GPU** parallelise les matrices. Le **transfer learning** reutilise un cerveau deja forme. Les LLM sont du deep learning langage + alignement + usage (prompt, RAG, agents).

Chez DanielCraft, on resume autrement : matcher le probleme, reutiliser avant de reentrainer, mesurer sur le terrain, garder un humain quand le risque monte.

### ★ A RETENIR

Carte DL : neurone -> couches -> activation -> backprop -> CNN / RNN / transformer -> overfitting -> GPU -> transfer -> LLM.

### Ce que ce n'est pas

Ce n'est pas une checklist a reciter sans exemple. Ce n'est pas "tu peux tout deployer demain". Ce n'est pas non plus "le ML classique est mort". Sur un petit tableau, reviens au livre machine learning. Sur une image ou du langage riche, tu as maintenant les bons mots.

Tache claire -> assez de donnees / transfert -> architecture adaptee -> loss et metriques -> validation -> regularisation -> deploiement mesure -> surveillance. Prefere reutiliser avant de reentrainer le monde. Ines a cette boucle sur une feuille A4. Lea l'a collee dans ses briefs. Max l'a simplifiee en trois questions : "ca sert ?", "on mesure quoi ?", "si ca se trompe ?".

### Petite histoire

Sam demande a un eleve de tout raconter en deux minutes. L'eleve bloque sur backprop. Il relit le chapitre 5, recommence, passe. La carte n'est solide que si elle passe a l'oral. Chez DanielCraft, l'oral est le vrai test : si tu peux l'expliquer a un non-tech, c'est entre.

## Pont vers la pratique

Si tu utilises déjà un chat : tu manipules un transformer aligne. Si tu classes des images : tu toucheras CNN / transfer. Si tu as un CSV : reviens au ML. La bonne stack est celle qui matche le problème. Ines mélange parfois vision (CNN) et doc utilisateur (LLM) : deux briques, un protocole de vérification pour chacune.

## Erreur classique

Vouloir tout retenir comme une liste de courses sans scène concrète. Autre piège : passer aux ateliers sans avoir une tâche personnelle en tête. Les ateliers collent mieux à ta vie si tu apportes ton problème.

### ⚠ ATTENTION

Une carte sans exemple personnel devient du papier peint. Ajoute le tien.

## En vrai

Ferme le livre (ou l'écran). Sur une feuille blanche, reconstitue la carte en TES mots. Une seule face. Interdit de recopier les titres mot à mot : reformule.

## A toi

Carte mentale une page : un exemple vision et un exemple texte. Relie chaque boîte à une décision que tu pourrais prendre cette semaine (même minuscule).

## Anti-patterns à garder en tête

LLM pour classer trois catégories sur un CSV de vingt colonnes. CNN from scratch avec quarante images. Agent autonome le jour 1. Transformer "parce que c'est moderne" là où un modèle simple suffit. Note tes anti-patterns personnels à côté de la carte.

## Suite immédiate

Atelier intuition (neurone et couches), atelier plan CNN, atelier transformer / usage LLM, puis choix d'architecture, limites, bonnes pratiques, quiz. Tu n'empiles plus seulement des couches de neurones. Tu empiles des gestes. C'est le but.

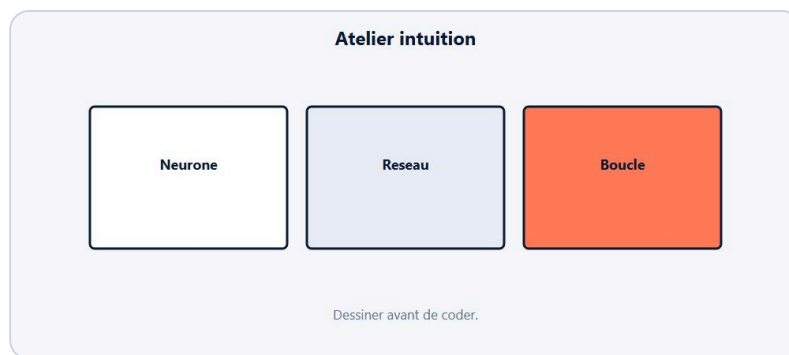
## Checklist orale express

En moins de deux minutes : qu'est-ce qu'un neurone ? Pourquoi activer ? Comment apprend-on ? Pourquoi CNN sur image ? Pourquoi attention sur texte ? Qu'est-ce qui overfitte ? Pourquoi GPU ? Pourquoi transfer ? Lien LLM ? Si une case manque, tu sais où revenir. Lea fait cette checklist avant chaque atelier client. Ines avant chaque demo.

## Ce que tu peux déjà faire demain

Choisir une baseline simple sur un CSV. Planifier un transfer vision. Briefer un LLM avec faits et anti-invention. Dire non à un from scratch injustifié. Ces quatre gestes valent plus qu'une citation de papier. Chez DanielCraft, la carte sert à agir petit et juste.

## Chapitre 14 - Atelier : intuition neurone et couches



Atelier : neurone, reseau, boucle d'apprentissage.

Objectif : solidifier l'intuition sans code obligatoire. Duree visee : environ 30 minutes. Tu sors avec un dessin explicable a un collegue non tech - le meilleur test de comprehension selon DanielCraft.

Cet atelier n'est pas un examen. C'est une gymnastique. Ines le fait avant chaque nouveau projet vision pour "remettre les briques a plat". Lea s'en sert pour briefer. Sam le transforme en exercice de classe.

### ★ A RETENIR

Si tu peux expliquer ton dessin et la boucle d'apprentissage a quelqu'un, l'atelier a reussi.

### Ce que ce n'est pas

Ce n'est pas un TP PyTorch obligatoire. Le code optionnel existe pour les curieux ; le coeur est papier + oral. Ce n'est pas non plus "faire joli". Un schema sale et juste bat un schema parfait et creux.

Un neurone : entrees, poids, biais, activation, sortie. Une couche : plusieurs neurones en parallele. Un petit reseau : empilement. Une boucle : batch, avant, loss, arriere, maj des poids, repeter. Tu vas faire vivre ces quatre phrases.

### Exercices (dans l'ordre)

- 1) Dessine un neurone a 3 entrees, avec poids, biais, activation. Invente des nombres simples et calcule a la main une sortie **ReLU** ( $\max(0, \text{somme})$ ). Ne cherche pas la beaute ; cherche le geste.
- 2) Empile 2 couches denses sur papier (tailles 3 -> 4 -> 1). Explique oralement le passage avant : que recoit chaque etage, que produit-il ?
- 3) Ecris la boucle d'apprentissage en 6 etapes. Chronometre une version orale de 40 secondes.
- 4) Identifie pour ton projet (ou celui d'Ines : pieces sur photo) : entree brute, representation utile imaginee, sortie. Trois fleches suffisent.
- 5) Option code : un mini reseau toy dans un notebook (meme 20 lignes) pour voir la loss baisser sur une fonction simple. Seulement si tu en as envie et le temps.

### ◆ ASTUCE

Fais l'exercice 1 au stylo, pas au clavier. La main ralentit assez pour que le concept accroche.

## Petite histoire

Max a refuse d'abord : "je suis pas developpeur". Lea lui a fait faire le neurone parapluie (pluie, vent, distance). En dix minutes, il expliquait poids et seuil a son apprenti. Le code peut attendre. L'intuition, non.

## Erreur classique

Sauter au code optionnel sans dessin. Ou recopier un schema internet sans nombres. Ou expliquer avec des mots savants pour masquer un trou. Si tu bloques, reviens au chapitre 2 et 5, puis recommence l'oral.

### ⚠ ATTENTION

L'atelier echoue si tu ne parles a personne. Trouve un humain, meme presse, meme distant.

## En vrai

Appelle ou assieds quelqu'un. Montre le dessin. Demande : "reformule la backprop en une phrase". Si la personne y arrive apres toi, tu as transmis. C'est le livrable invisible.

## Livrable

Une feuille dessinee + 10 lignes d'explication que tu pourrais envoyer a un collegue non tech. Garde la feuille : l'atelier CNN s'appuiera sur la meme discipline de clarte.

## A toi

Coche : neurone calcule, reseau 3-4-1 explique, boucle orale OK, fleches projet OK, explication lue a quelqu'un. Une case vide = cinq minutes de plus, pas un abandon.

## Variante Ines

Remplace le parapluie par "piece A / pas piece A". Entrees inventees : score contour, score texture, score trou. Meme neurone. Plus proche de son appli. Adapte a ton domaine : le transfert d'intuition est le vrai sujet.

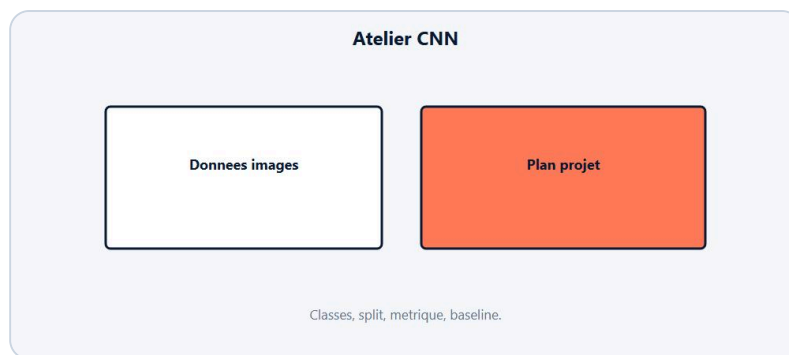
## Grille d'auto-evaluation

Apres l'atelier, note 1 a 5 : clarte du dessin, exactitude du calcul ReLU, fluidite de l'oral backprop, pertinence des fleches projet, qualite de l'explication non tech. Tout score sous 3 = cinq minutes de rattrapage cible. Sam utilise la grille en classe ; Lea en autofeedback.

## Piege du vocabulaire savant

Si tu dis "gradient stochastique" mais tu ne sais pas raconter batch / avant / loss / arriere, tu te caches. Inverse la priorite : histoire simple d'abord, mot savant ensuite. Chez DanielCraft, le jargon arrive en recompense de la clarte, pas en camouflage.

## Chapitre 15 - Atelier : CNN et transfer learning (plan de projet)



Atelier CNN : plan de projet vision.

Objectif : ecrire un plan de projet vision realiste. Duree visee : environ 40 minutes. Tu n'as pas besoin d'avoir entraine pour valider le plan. Chez DanielCraft, un plan honnete bat un notebook theatral.

Ines utilise exactement cette grille pour ses pieces detachees. Lea s'en sert pour cadrer un prestataire. Toi, tu l'adaptes a ton jeu d'images - meme imaginaire, du moment qu'il est concret.

### ★ A RETENIR

Plan CNN = classes, donnees, conditions reelles, preentraine, augmentation, split, metriques, anti-overfitting, go/no-go, calcul.

### Ce que ce n'est pas

Ce n'est pas un entrainement complet. Ce n'est pas une promesse de 99 %. Ce n'est pas non plus "collecter 50 images et deployer demain". Si tu as moins de 50 images au total, reste sur proof of concept et transfer learning agressif, ou reconsidere la faisabilite.

Tu construis un dossier de decision. Chaque section empeche un mensonge courant : "on verra bien", "le modele saura", "les photos studio suffisent". Le go/no-go signe a l'avance te protege du deploiement par enthousiasme.

### Etapes (dans l'ordre)

- 1) Definir 3 a 10 classes d'images. Noms clairs, frontieres discutees (qu'est-ce qui est ambigu ?).
- 2) Estimer combien de photos par classe tu peux collecter cette semaine / ce mois.
- 3) Decrire les conditions reelles : eclairage, flou, telephone, angle, mains dans le cadre, huile, pluie.
- 4) Choisir un modele preentraine a adapter (famille CNN ou vision transformer, peu importe le detail : l'esprit transfer).
- 5) Plan d'augmentation de donnees : liste de transformations qui existent en production sans casser le label.
- 6) Split train / val / test. Qui touche au test ? Ideallyment presque personne jusqu'a la fin.
- 7) Metriques : accuracy si utile, mais surtout erreurs couteuses (faux positif vs faux negatif selon le metier).



8) Plan anti-overfitting : early stopping, dropout, moins de capacite, plus de diversite, gel de couches...

9) Critere go/no-go avant deploiement. Une phrase signee.

10) Idee GPU : local, cloud, ou seulement inference API.

Esquisse code (a coller / adapter quand tu codes pour de vrai) :

```
# Plan minimal : transfer + early stopping (pseudo)
# 1. backbone = resnet18(weights="DEFAULT") ; geler
# 2. remplacer fc par Linear(..., n_classes)
# 3. DataLoader train/val avec augmentations realistes
# 4. boucle epochs : loss, val ; early stopping patience=5
# 5. evaluer UNE FOIS sur test terrain reserve
```

#### ♦ ASTUCE

Ecris le go/no-go AVANT de regarder un premier score. Sinon tu deplaces la barre.

### Petite histoire

Ines a ecrit : "si recall sur pieces critiques < seuil X sur test terrain, on n'automatise pas ; humain valide". Un premier modele a echoue sous le seuil. Elle a ameliore les donnees, pas le marketing. Max a applique la meme idee a un "detecteur de fuite sur photo" trop frele : abstention > fausse alerte permanente.

### Erreur classique

Remplir le plan avec des chiffres inventes pour faire serieux. Ou oublier les conditions terrain. Ou choisir from scratch par snobisme. Ou ne jamais ecrire le no-go.

#### ⚠ ATTENTION

Sans critere d'arret metier, tu deploieras par fatigue ou par slide.

### En vrai

Prends un timer 40 minutes. Remplis les 10 etapes au brouillon. Interdiction d'ouvrir un framework avant la fin du document.

### Livable

Document "plan CNN" de 1 a 2 pages. Titre, classes, volumes, conditions, modele preentaine, augmentation, split, metriques, anti-overfitting, go/no-go, calcul. Envoie-le a un pair pour contradiction.

### A toi

Signe ton go/no-go. Formule type : "si la metrique X sur test terrain < Y, on n'automatise pas, on garde un humain dans la boucle". Date. Initiales.

### Conseil DanielCraft

Si les labels sont ambigus, arrete-toi a l'etape 1 et clarifie avec un expert metier. Aucun CNN ne sauve une taxonomie pourrie. Sam le fait dire a ses eleves avant tout telechargement de backbone.

## Exemple Ines (rempli partiellement)

Classes : 6 types de pieces. Volume vise : 80 photos / classe, telephone chantier. Conditions : neon, graisse, angle 30-60 degrees. Modele : CNN preentraine ImageNet-like. Augmentation : rotation legere, luminosite, crop ; pas de flip si le sens compte. Split 70/15/15. Metrique : recall sur pieces critiques + revue humaine si score bas. Go/no-go : recall terrain < seuil => pas d'automatisation. GPU : cloud ponctuel. Copie la structure, change les chiffres.

## Revue par un pair

Envoie le plan a quelqu'un qui cherchera les trous : classes floues, volumes irrealistes, oubli du terrain, absence de no-go. Une contradiction amicale aujourd'hui evite un deploiement honteux demain.

## Chapitre 16 - Atelier : transformer et usage LLM



Atelier transformer : attention, RAG, risques.

Objectif : relier architecture et gestes d'usage. Duree visee : environ 35 minutes. Tu sors avec une fiche "du transformer a mon usage" : schema d'attention, protocole RAG manuel, regles de cout et de verification.

Chez DanielCraft, comprendre le moteur sans protocole d'usage, c'est laisser le frein au garage. Cet atelier pose le frein.

### ★ A RETENIR

Attention expliquee + contexte budgete + RAG manuel + temperature choisie + risques listes = usage adulte d'un LLM.

### Ce que ce n'est pas

Ce n'est pas un fine-tune obligatoire. Ce n'est pas "construire un agent autonome". Ce n'est pas non plus un concours de prompts miracles. C'est relier ce que tu as appris au chapitre 8 et 12 a des gestes que tu peux faire aujourd'hui.

Tu as un moteur attentionnel avec une fenetre de contexte limitee. Tu decids quoi mettre dans la fenetre, comment chercher des extraits, comment regler l'aleas, comment verifier. Lea fait ca pour ses propositions clients. Ines pour sa doc. Sam pour ses quiz. Max pour ses mails - avec interdiction d'inventer un delai.

### Etapes (dans l'ordre)

- 1) Explique l'attention avec ton propre exemple de phrase (pronom, reference, consigne metier). Ecris 8 lignes max.
- 2) Mesure mentalement une limite de contexte : que coupes-tu dans un long PDF ? Ordre de priorite.
- 3) Concevoir un mini-RAG : 5 documents (memes fictifs), une question, l'extrait a coller, une consigne anti-invention.

Correction type de mini-RAG manuel :

```
docs = {
    "tarif": "Prestation diagnostic : 120 EUR HT, déplacement inclus sous 20 km.",
    "delai": "Delai moyen annonce : 10 jours ouvres apres validation du devis.",
}
question = "Quel est le prix du diagnostic ?"
extrait = docs["tarif"] # tu choisis l'extrait ; le modele ne cherche pas seul ici

prompt = f"""Reponds UNIQUEMENT avec l'extrait.
Si l'info manque, dis "je ne sais pas".
Extrait:
{extrait}
Question: {question}
"""
print(prompt)
```

- 4) Deux temperatures mentales sur la meme tache (strict vs creatif) et une petite grille d'evaluation (utile / faux / flou).
- 5) Decrire un fine-tuning que tu NE feras pas maintenant, et pourquoi prompting / RAG suffisent pour l'instant.
- 6) Lister 3 risques : donnees sensibles, hallucination, cout tokens (ou temps de verification).

#### ♦ ASTUCE

Le mini-RAG manuel (copier l'extrait toi-meme) enseigne mieux que trois slides sur les vector stores.

### Petite histoire

Lea a teste le meme brief "strict" puis "creatif" pour un mail artisan. Le creatif a invente une promo. Le strict a demande le prix manquant. Elle a garde le strict pour l'envoi, le creatif pour brainstorm prive. Sam fait le meme exercice avec ses eleves : choisir la temperature, c'est choisir un risque.

### Budget tokens

Estime un cout mensuel grossier : questions / jour x tokens moyens x prix. Ajoute le cout de tes erreurs (temps de verification, risque metier). Decide si un modele plus petit suffit pour 80 % des cas. Ines reserve le modele "lourd" aux cas ambigus ; le reste passe au leger.

### Erreur classique

Coller tout le disque dans le contexte. Ou faire un "RAG" sans verifier que l'extrait repond a la question. Ou annoncer un fine-tune pour impressionner alors qu'un bon brief suffit. Ou ignorer le cout de verification humaine.

#### ⚠ ATTENTION

Un extrait mal choisi dans un RAG produit une reponse fluide et fausse. Le fluide empire le piege.

### En vrai

Fais le mini-RAG sur papier : question, extrait, consigne "n'utilise que l'extrait, dis inconnu sinon". Execute dans ton outil. Compare a la meme question sans extrait.

## **Livrable**

Fiche "du transformer a mon usage" avec : schema attention, protocole RAG manuel, deux reglages de temperature, budget tokens, 3 risques. Une page recto volait.

## **A toi**

Coche les 6 etapes. Puis ecris la phrase : "je fine-tune seulement si ..." avec un critere mesurable. Si tu ne trouves pas de critere, tu ne fine-tune pas.

## **Variante Max**

Remplace le PDF par un devis et une fiche produit. Meme atelier. Interdits : inventer delai, prix, garantie. C'est du deep learning applique a la confiance client.

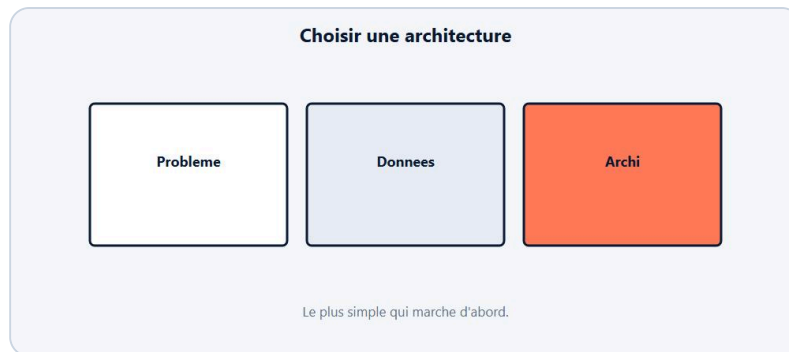
## **Exemple de mini-RAG (modele)**

Question : "Quel delai max pour la piece B ?". Document 3 : "Piece B : delai annonce 5 jours ouvres, hors rupture.". Consigne : "Reponds seulement avec l'extrait. Si absent, dis inconnu.". Sans extrait, le modele invente parfois 48 h. Avec extrait, il cite 5 jours. Max garde cet exemple scotche au-dessus de son ecran.

## **Grille d'evaluation rapide**

Pour chaque reponse : utile / faux / flou / danger donnees. Trois exemples suffisent pour sentir la temperature et le brief. Lea exige cette grille avant d'industrialiser un usage chat en entreprise.

## Chapitre 17 - Choisir une architecture (carte de decision)



Choisir une architecture selon le probleme.

Tu n'as pas a inventer une architecture nouvelle. Tu as a choisir une **famille** adaptee. Chez DanielCraft, le prestige d'un papier ne paie pas les faux positifs clients. On part du probleme, on descend vers le moteur - jamais l'inverse.

Ce chapitre consolide ce que tu as traverse : CNN, RNN, transformers, transfer, LLM, ML classique. C'est une carte de decision, pas un catalogue de marques.

### ★ A RETENIR

Pars du probleme et des contraintes (donnees, calcul, risque, latence). L'architecture suit.

### Ce que ce n'est pas

Ce n'est pas "toujours un LLM". Ce n'est pas "toujours un CNN". Ce n'est pas non plus une verite eternelle : en 2026 les familles evoluent, les principes de matching restent. Et ce n'est pas un permis d'ignorer une baseline simple.

Petit tableau numerique : ML classique d'abord. Images : CNN ou vision transformer preentraine + transfer. Texte generation / chat : LLM existant + prompt / RAG. Series temporelles : modeles specialises ou approches mixtes ; ne force pas un LLM partout. Audio : modeles parole / audio preentraines. Multi-taches complexes : parfois pipelines (vision puis regles puis LLM). Ines range ses sujets dans ces cases avant d'ouvrir un repo.

### Questions de decision

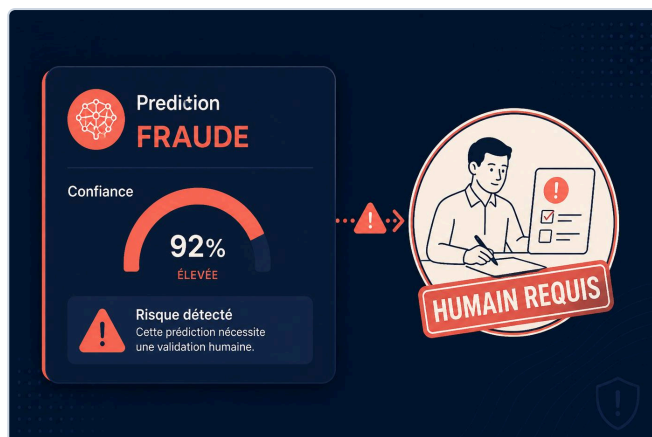
Combien de donnees labellisees ? Quel budget calcul ? Quelle latence acceptable ? Quel besoin d'interpretabilite ? Quel risque d'erreur ? Existe-t-il un modele fondation proche ? Peut-on resoudre sans deep learning ? Lea pose ces questions en reunion ; les reponses changent le devis plus surement qu'un buzzword.

### ♦ ASTUCE

Si tu ne peux pas repondre a "que se passe-t-il si le modele se trompe ?", tu n'es pas pret a choisir l'architecture.

## Anti-patterns

LLM pour classer 3 categories sur un CSV de 20 colonnes. CNN from scratch avec 40 images. Agent autonome le jour 1. Transformer "parce que c'est moderne" sur une serie temporelle ou un modele simple suffit. Pipeline opaque a trois etages sans test de chaque brique. Note tes anti-patterns personnels : ils reviennent.



Exemple : une prediction confiante peut exiger un humain.

## Petite histoire

Un client a demande a Lea "un transformer sur nos exports comptables". Elle a montre qu'un modele tabulaire + regles battait un prototype LLM en cout, latence, et auditabilite. Le client a garde le LLM pour rediger des commentaires, pas pour scorer. Deux outils, deux jobs. Max a vecu l'inverse : pour reconnaitre une piece sur photo, le tableur ne suffisait pas ; le CNN transfer oui.

## Erreur classique

Partir d'une architecture parce qu'elle est a la mode. Autre piege : copier l'architecture d'un concurrent sans avoir ses donnees ni son risque. Troisieme : melanger trois familles dans un pipeline sans pouvoir expliquer le role de chacune.

### ⚠ ATTENTION

Une architecture impressionnante mal evaluee reste une dette. Choisis le minimum qui prouve la valeur.

## En vrai

Prends deux problemes de ton monde. Pour chacun, remplis : famille choisie, raison, alternative, critere d'echec. Dix minutes chrono.

## A toi

Remplis la carte pour 2 problemes. Architecture choisie + raison + alternative + risque principal. Garde-la a cote du plan CNN et de la fiche transformer.

## Pipelines mixtes

Parfois le bon systeme n'est pas "un" reseau. Vision pour classer, regles pour seuils, LLM pour expliquer a un humain. Teste chaque brique, puis l'ensemble. Sam insiste : un pipeline non teste multiplie les hallucinations operationnelles. Chez DanielCraft, on prefere trois briques claires a une boite noire unique adulee.

## Lien avec limites et bonnes pratiques

Choisir une architecture, c'est aussi accepter ses limites (prochain chapitre) et adopter des pratiques sobres (chapitre suivant). La carte de decision n'est complete que branchee sur le risque et la discipline d'execution.

## Scene de reunion

"On veut du deep learning." "Quel probleme ?" "Classer 4 types de tickets texte." "Combien de labels ?" "300." "Baseline TF-IDF ou modele texte preentraîne léger d'abord. LLM complet ensuite seulement si besoin d'explication riche." La reunion dure douze minutes. Le prestige du papier n'a pas parle. Chez DanielCraft, c'est une belle reunion.

## Matrice express (a recopier)

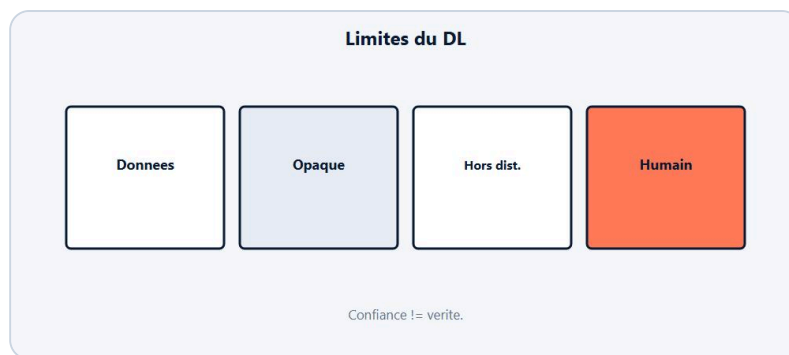
Lignes : tableau, image, texte court classif, texte generation, serie temporelle, audio. Colonnes : famille candidate, preentraîne ?, besoin GPU, risque erreur, alternative simple. Remplir en 15 minutes change une reunion. Ines l'a fait ; le partenaire a arrete de demander "du GPT sur les colonnes Excel".

## Quand mixer

Vision pour detecter, regle pour seuil, LLM pour expliquer. Chaque fleche du pipeline a un test. Si tu ne peux pas tester une fleche, tu ne la deploies pas seule. Chez DanielCraft, le mixte assume bat le monolithe mystique.



## Chapitre 18 - Limites du deep learning



Limites : donnees, opaquete, hors distribution, abstention.

Le deep learning est puissant et borne. Il demande des donnees, du calcul, de l'ingenierie. Il peut etre opaque. Il peut etre fragile hors distribution. Il peut encoder des biais. Il peut couter cher en inference a grande echelle. Il peut donner une illusion de comprehension. Chez DanielCraft, voir les limites n'est pas du pessimisme : c'est du professionnalisme.

Ce chapitre ne tranche pas tous les debats ethiques. Il exige que tu les voies assez pour decider en adulte.

### ★ A RETENIR

Puissance ≠ comprehension. Prevois repli, abstention, mesure hors distribution, et couts reels.

### Ce que ce n'est pas

Ce n'est pas une raison d'abandonner. Ce n'est pas non plus une excuse pour tout automatiser "parce que le score labo est beau". Ce n'est pas "les modeles simples n'ont pas de limites" : ils en ont d'autres. Et ce n'est pas uniquement un sujet recherche : en production, les limites arrivent en ticket support.

### Ce qu'il ne remplace pas

La definition du probleme. La qualite des labels. La decision ethique. La responsabilite legale. La connaissance terrain. Les modeles simples quand ils suffisent. Les processus humains de verification. Ines peut avoir un excellent CNN ; elle reste responsable si une mauvaise piece passe. Lea le rappelle dans ses CGV de projet. Max le sent dans sa garantie chantier.

Change l'eclairage, l'accent, le format de document, la saison : les performances chutent parfois. Les adversaires peuvent tromper certains modeles avec des perturbations. En production, prevois des replis : regles, humain, **abstention**. Parfois la meilleure sortie est "je ne sais pas / humain requis". Concois un seuil d'abstention. En vision comme en langage, l'abstention sauve des catastrophes.

### ♦ ASTUCE

Ecris une phrase d'abstention pour ton projet avant d'ecrire une phrase de deployment.

## Environnement et cout

Entraîner et servir de gros modèles a un impact énergie / argent. Utilise le plus petit modèle qui fait le job. Préfère réutiliser. Mesure avant d'empiler. Sam fait calculer à ses élèves un ordre de grandeur : pas pour culpabiliser, pour choisir. Lea refuse les devis "on scale d'abord, on mesure après".

## Petite histoire

Ines a déployé sans seuil d'abstention. Le modèle classait "avec confiance" des pièces hors catalogue vers la classe la plus proche. Clients mécontents. Elle a ajouté un score minimum + file humaine. Le taux d'automatisation a baissé. La confiance client a monté. Chez DanielCraft, on appelle ça une victoire.

## Biais et angles morts

Les données d'entraînement portent des déséquilibres. Un modèle vision peut moins bien marcher sur certains éclairages, peaux, lieux, appareils. Un LLM peut reproduire des stéréotypes ou des angles culturels. Tu n'as pas à tout résoudre seul. Tu as à tester des sous-groupes, écouter les échecs, documenter. Ignorer n'est pas neutre.

## Erreur classique

Croire que "plus de deep learning" efface les limites. Autre piège : cacher les échecs pour protéger le score. Troisième : confondre opacité du modèle et absence de responsabilité humaine.

### ⚠ ATTENTION

Utiliser un outil puissant sans regarder ses effets collatéraux, ce n'est pas de la neutralité technique. C'est une décision. Assume-la.

## En vrai

Liste 5 limites qui s'appliquent à ton projet. Pour chacune, une mitigation : humain, règle, plus de données, modèle plus petit, abstention, monitoring...

## A toi

Tableau 5 lignes : limite / impact / mitigation / responsable. Signe en bas. C'est ton mini plan de risque.

## Illusion de compréhension

Un texte fluide, une classe prédite, une heatmap jolie : rien de tout cela ne prouve une compréhension humaine. Ce sont des comportements utiles sous conditions. Le chapitre LLM l'a dit ; les limites le martellent. Garde l'humilité quand tu présentes à un non-tech : tu vends un outil borné, pas un oracle.

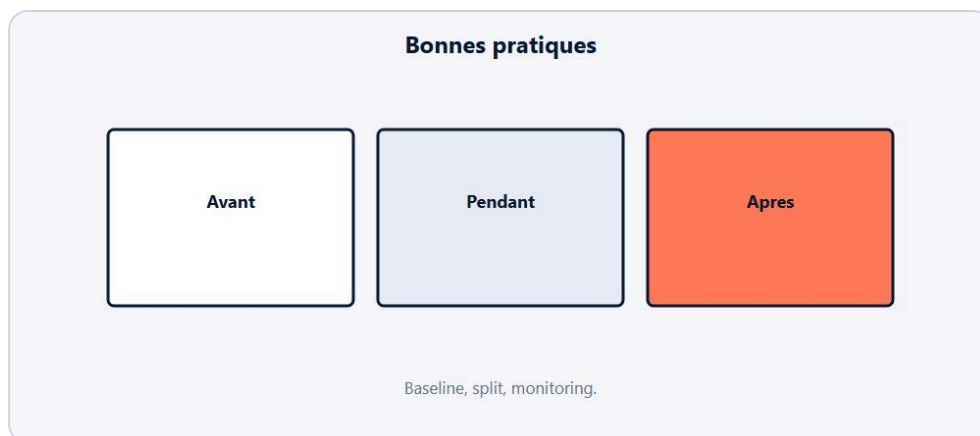
## Hors distribution : trois exemples

Photo de nuit alors que le train était de jour. Accent ou jargon absent du corpus. Format PDF scanné alors que le modèle a vu du texte propre. Dans chaque cas, le système peut répondre quand même - avec assurance. D'où abstention et monitoring. Sam collectionne ces exemples pour sa classe ; Ines pour son journal d'incidents.

## Limites legales et organisationnelles

Qui est responsable si le modele se trompe ? Quelles donnees a-t-on le droit d'envoyer a une API ? Qui relit ? Ces questions ne sont pas "hors sujet technique". Elles bornent le deploiement autant qu'un manque de VRAM. Lea les met en page 1 du cadrage.

## Chapitre 19 - Bonnes pratiques deep learning



Bonnes pratiques : avant, pendant, apres.

Les bonnes pratiques ne sont pas du folklore. Ce sont des garde-fous contre l'enthousiasme, l'oubli, et le "ca marche sur mon laptop". Chez DanielCraft, on les range en avant / pendant / apres, plus un mot sur les pipelines mixtes et la reproductibilite minimale.

Tu peux les transformer en checklist de 12 cases. C'est l'exercice de fin. L'objectif n'est pas la perfection documentaire : c'est eviter les erreurs deja payees par d'autres.

### ★ A RETENIR

Valider le besoin, reutiliser, mesurer, regulariser, inspecter les echecs, versionner, monitorer. Sobriete > theatre.

### Ce que ce n'est pas

Ce n'est pas une norme ISO a imprimer pour faire joli. Ce n'est pas non plus "ralentir pour ralentir". C'est accelerer juste : moins de retours catastrophiques, plus de cycles utiles. Et ce n'est pas reserve aux grandes equipes : Ines seule peut cocher l'essentiel.

### Avant

Valider que le deep learning est necessaire. Chercher un modele preentraîne. Estimer donnees et calcul. Definir metriques et risques. Preparer un set de validation propre. Ecrire un go/no-go. Clarifier les labels avec le metier. Lea refuse de demarrer un sprint vision sans ces points ecrits.

### Pendant

Commencer petit. Logger loss et metriques. Early stopping. Regulariser. Inspecter les echecs (les vrais fichiers, pas seulement la moyenne). Ne pas toucher au test final trop souvent. Documenter hyperparametres et versions. Changer une chose a la fois. Comparer a une baseline. Arrêter quand la validation stagne. Sam appelle ca "experimentation sobre" - le mot compte moins que le geste.

#### ♦ ASTUCE

Garde un journal d'experiences en 5 colonnes : hypothese, changement, metrique, decision, date. Une demi-heure de rangement economise des jours.

## Apres

Evaluer hors distribution. Monitorer. Prevoir reentrainement. Versionner. Separer experiences jouets et systeme de production. Informer les utilisateurs des limites. Prevoir un repli humain. Mesurer le cout d'inference reel, pas seulement le cout d'entrainement. Max veut savoir ce qui se passe quand le telephone change ; Ines aussi.

## Usage LLM couple

Si ton systeme mele CNN + LLM + regles, teste chaque brique, puis l'ensemble. Un pipeline opaque a trois etages multiplie les hallucinations operationnelles. Lea exige un compte-rendu d'erreur par etage. Chez DanielCraft, "ca a l'air fluide de bout en bout" n'est pas un critere d'acceptation.

## Petite histoire

Ines a perdu une apres-midi a "ca marche plus". Pas de graine, pas de version donnees, pas de config sauvee. Elle a institue une reproductibilite minimale : graine, version code, version donnees, config, commit, metriques. Depuis, les regressions se discutent avec des faits. Sam a vole la checklist pour sa classe.

## Reproductibilite minimale

Graine, version code, version donnees, config, commit, metriques. Sans ca, tu ne sauras pas pourquoi "ca marche plus". Ce n'est pas du luxe enterprise. C'est de l'hygiene. Ajoute les droits / licences des modeles fondation si tu transfers.

## Erreur classique

Tout faire a la fois. Ne jamais inspecter un echec. Optimiser le test. Oublier le monitoring apres la demo. Documenter seulement quand un manager crie. Ou croire que les bonnes pratiques ralentissent les "vrais" talents.

#### ⚠ ATTENTION

Une demo sans protocole n'est pas un produit. C'est une anecdote couteuse.

## En vrai

Cree ta checklist 12 cases a partir de ce chapitre (avant / pendant / apres melanges). Imprime-la ou epingle-la.

## A toi

Coche la checklist sur ton prochain essai, meme toy. Une case refusee = une phrase "pourquoi j'accepte le risque". Si tu ne peux pas l'ecrire, coche autrement : en faisant le geste.

## **Culture de communication**

Apprends a dire "non" a un modele inutile. Apprends a dire "pas encore" quand les labels manquent. Apprends a dire "voici les limites" quand tu presentes un score. Cette honnetete te rend plus credible que n'importe quel jargon. C'est la derniere bonne pratique, et souvent la plus rentable.

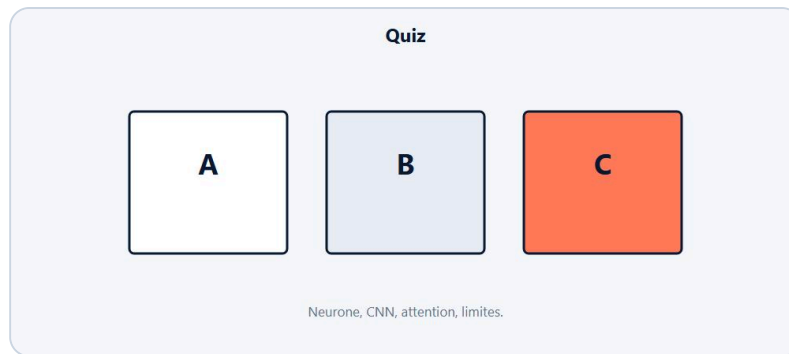
## **Les 12 cases (modele)**

1) Besoin DL valide. 2) Baseline / preentraine cherche. 3) Metriques et risques ecrits. 4) Val propre. 5) Go/no-go. 6) Journal d'experiences. 7) Early stopping. 8) Inspection d'echecs. 9) Test rare. 10) Versions (code/donnees/config). 11) Monitoring prevu. 12) Limites communiquees. Coche. Si tu sautes, ecris pourquoi.

## **Separer jouet et production**

Un notebook d'exploration peut etre sale. Un systeme qui touche un client ne peut pas heriter du sale sans rituel. Promotion volontaire : code range, metriques, rollback, responsable. Chez DanielCraft, confondre les deux est le classique qui coute le plus cher apres la demo.

## Chapitre 20 - Quiz final



Quiz : verifier le socle sans notes.

Ce quiz n'est pas un piege. C'est un miroir. Chez DanielCraft, on prefere dix questions honestes a cinquante QCM pinailleurs. Lis chaque enonce, choisis A, B ou C, puis verifie. Si tu rates, relis le chapitre vise - pas tout le livre en diagonale.

Ines a passe ce quiz avant de presenter son plan CNN a un partenaire. Lea s'en sert pour cadrer un atelier client. Toi, fais-le stylo en main, sans scroll vers les corriges.

### ★ A RETENIR

Le quiz verifie le socle : profondeur, activation, backprop, CNN, transformers, overfitting, GPU, transfer, LLM, methode.

### Avant de commencer

Ferme tes notes. Garde seulement une feuille blanche pour tes lettres de reponse. Chronometre 12 minutes si tu veux jouer le jeu "a chaud". Sinon, prends le temps - la comprehension bat la vitesse.

### Questions

1. 1. Le "deep" de deep learning renvoie surtout a :

- A) La couleur sombre des GPU
- B) Des reseaux a plusieurs couches
- C) Une voix grave

1. 2. Une activation non lineaire sert a :

- A) Decorer les graphiques
- B) Permettre aux couches empilees d'approximer des motifs complexes
- C) Remplacer les donnees

1. 3. La backprop, en idee :

- A) Ajuste les poids a partir de l'erreur
- B) Efface le disque dur
- C) Genere des images sans modele

1. 4. Un CNN est particulièrement utile pour :

- A) Les filtres locaux sur images
- B) Remplacer toute base de données
- C) Facturer les API

1. 5. Les transformers reposent surtout sur :

- A) L'attention entre positions / tokens
- B) Une unique droite de régression
- C) Un seul neurone

1. 6. L'overfitting DL se manifeste souvent par :

- A) Train excellent, validation faible
- B) Train faible, validation parfaite toujours
- C) Absence totale d'erreur humaine

1. 7. Un GPU aide surtout parce que :

- A) Il parallélise de gros calculs de matrices
- B) Il écrit tes prompts
- C) Il anonymise les données tout seul

1. 8. Le transfer learning, c'est :

- A) Réutiliser un modèle préentraîné pour l'adapter
- B) Transférer des fichiers par FTP uniquement
- C) Interdit en 2026

1. 9. Un LLM est :

- A) Sans aucun lien avec le deep learning
- B) Un modèle de deep learning (souvent transformer) pour le langage
- C) Un type de câble

1. 10. Avant d'entraîner un gros réseau from scratch, DanielCraft recommande souvent :

- A) De regarder si un modèle préentraîné / une API / un modèle simple suffit
- B) D'acheter dix GPU sans question métier
- C) D'ignorer la validation

### **Petite histoire de correction**

Sam fait passer le quiz à sa classe. Ceux qui ratent la 6 et la 10 ont souvent "joué" avec un gros modèle trop tôt. Il les renvoie aux chapitres overfitting et transfer, puis au plan CNN. Lea rate parfois la 5 si elle est fatiguée - elle relit l'exemple du pronom "il", et ça revient. Normal : le quiz sert à cibler, pas à humilier.

### **Corriges**

1-B, 2-B, 3-A, 4-A, 5-A, 6-A, 7-A, 8-A, 9-B, 10-A.



9/10 ou plus : bon socle. 7-8 : relis les items rates et refais l'oral du chapitre 13. Moins de 7 : reprends couches, backprop, CNN / transformers, transfer, lien LLM - dans cet ordre.

#### **ATTENTION**

Rester sur 10/10 sans pouvoir expliquer oralement n'est pas le but. Le bonus oral compte autant.

### **Bonus oral**

Sans notes, explique à un ami : neurone, couche, backprop, CNN, transformer, transfer learning, lien LLM. Si tu bloques sur un, relis le chapitre, recommence. Chronomètre deux minutes. Ines le fait avant chaque demo client.

### **En vrai**

Prends une question ratee. Ecris en cinq lignes pourquoi la bonne réponse est bonne, avec ton exemple métier. Coller le corrigé ne suffit pas.

### **A toi**

Note ton score, les numéros rates, le chapitre à relire, et la date du prochain oral bonus. Une ligne suffit. Puis passe au bravo - ou reviens d'abord boucler les trous.

### **Pourquoi ces distracteurs**

Les mauvaises réponses jouent des confusions réelles : GPU = magie, LLM sans deep learning, overfitting inverse, transformer = régression. Si tu as hésité, note la confusion : elle reviendra en réunion. Le quiz sert aussi à ça.

### **Après le score**

Ecris trois phrases : ce que tu maîtrises assez pour expliquer, ce que tu dois relire, ce que tu vas pratiquer cette semaine. Ines envoie ces trois phrases à Lea comme engagement léger. Toi, envoie-les à ton futur toi.

## Chapitre 21 - Bravo



Bravo. Tu as une carte du deep learning.

Tu as maintenant une carte du deep learning : neurones, couches, activations, apprentissage par retour d'erreur, CNN, RNN en overview, transformers, overfitting, GPU, transfer learning, et le pont vers les LLM que tu utilises peut-être déjà tous les jours.

Chez DanielCraft, on croit à cette clarté : tu n'as pas besoin de prétendre que tu dérivés des gradients sur un tableau avant le café. Tu as besoin de savoir ce qui se passe sous le capot assez pour choisir, questionner, et ne pas acheter de la magie.

### ★ A RETENIR

Tu as une carte et des gestes. Empile des compétences évaluées, pas seulement des couches de neurones.

### Ce que tu emportes

Tu sais matcher un problème à une famille. Tu sais pourquoi la non-linéarité compte, comment l'erreur remonte, pourquoi un CNN aime le local, pourquoi un transformer aime le contexte, pourquoi l'overfitting frappe fort, pourquoi le GPU accélère, pourquoi le transfer learning est souvent le bon premier geste, et comment un chat LLM se branche sur tout ça. Tu as écrit des plans, des fiches, des go/no-go. Ce n'est plus seulement de la lecture.

Inès regarde son appli pièces : backbone préentraînée, tête adaptée, seuil d'abstention, monitoring. Lea briefe sans jargon panique. Max vérifie les délais qu'un LLM pourrait inventer. Sam fait passer la carte à l'oral. Quatre chemins, une même posture de pilote.

### Petite histoire

Le premier jour, "deep learning" sonnait comme une salle machine interdite. Aujourd'hui, tu peux dire : "ici un CNN transfer, là un LLM + RAG, là un modèle tabulaire suffit". Cette phrase seule change des réunions. Elle t'évite des devis absurdes. Elle t'ouvre des projets sobres. Bravo pour ça.

### La suite possible

- 1) Mini classification d'images par transfer learning, avec le plan du chapitre 15 exécuté pour de vrai.
- 2) Usage LLM + RAG évalué, avec la fiche du chapitre 16 et une grille d'erreurs.

3) Relire le livre machine learning pour metriques, biais, et baselines sur CSV.

Tu as le vocabulaire ; il te faut des cicatrices d'experiences - les bonnes. Choisis une suite petite, bornee, mesurable. Pas trois a la fois.

### **Erreur classique apres un livre**

Empiler les tutos sans projet. Ou lancer un agent autonome "parce que j'ai compris les transformers". Ou croire que le quiz 10/10 remplace le terrain. Le livre se termine ; la boucle DanielCraft continue : tache, mesure, repli, iteration.

#### **⚠ ATTENTION**

Le prochain pas utile est petit et evalue. Le prochain pas theatral peut attendre.

### **En vrai**

Ecris demain matin une seule action de 45 minutes : collecter 20 images, ou rediger un prompt + extrait RAG, ou refaire l'oral du chapitre 13 a quelqu'un. Une action. Pas une roadmap.

### **A toi**

Choisis ta suite (1, 2 ou 3). Note date, critere de succes, critere d'arret. Signe. C'est ton pont hors livre.

### **Remerciement utile**

Si ce livre t'a evite un from scratch inutile, un GPU loue trop tot, ou une confiance aveugle dans un chat fluide, il a fait son travail. Partage la carte, pas le jargon. Et souviens-toi : empile des couches de competence, pas seulement des couches de neurones.

Bravo.

### **Message a toi dans trois mois**

Si tu relis ce chapitre et que tu n'as toujours pas fait un petit projet evalue, le livre est reste du papier. Reprends le "A toi" du chapitre 1 et les livrables des ateliers. Choisis le plus petit geste encore non fait. Chez DanielCraft, on prefere un transfer mediocre mesure a une culture generale brillante et sterile.

### **Partager sans jargon**

Explique a un non-tech, en une minute : "des couches qui apprennent des motifs, on reutilise souvent un modele deja forme, on mesure sur le vrai terrain, on garde un humain si doute". Si tu y arrives, tu as gagne le vrai diplome de ce livre. Bravo encore.

Tu as les briques. Maintenant, construis.